Keyword Spotting and Audio Scene Classification

Marco Zennaro, PhD
The Abdus Salam International Centre for Theoretical Physics –Italy

Personal Assistant



There are an estimated **8.4 billion devices with AI personal assistants** (like Alexa, Siri, Google Assistant, etc.) in use globally in 2026.

Keyword Spotting v. General Speech Recognition

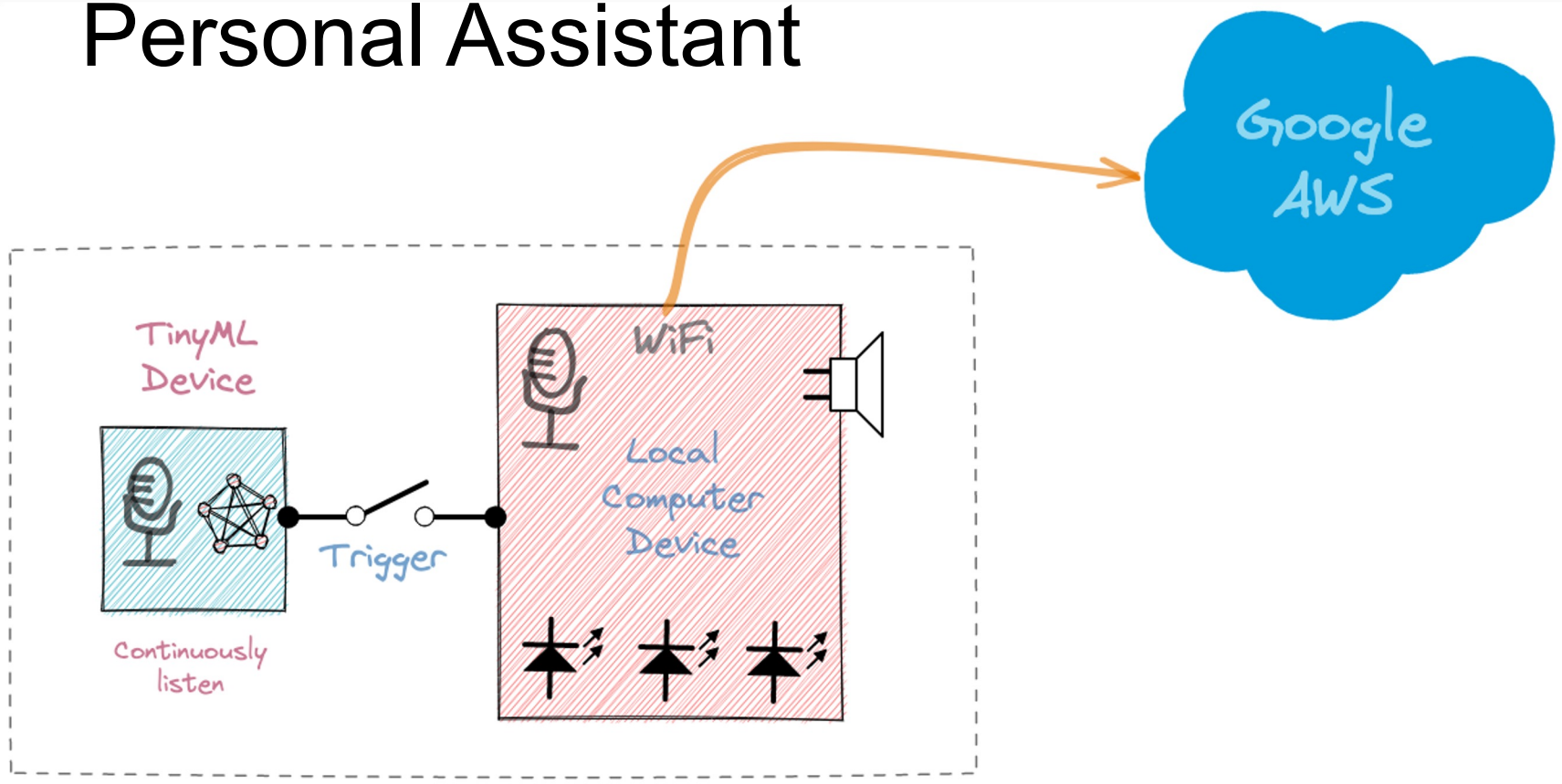
- **Keyword spotting** is one of the most successful examples of **TinyML**
  - Low-power, continuous, on-device
  - Common Voice SWTS* expands keyword spotting to more languages

    * **S**ingle **W**ord **T**arget **S**egment

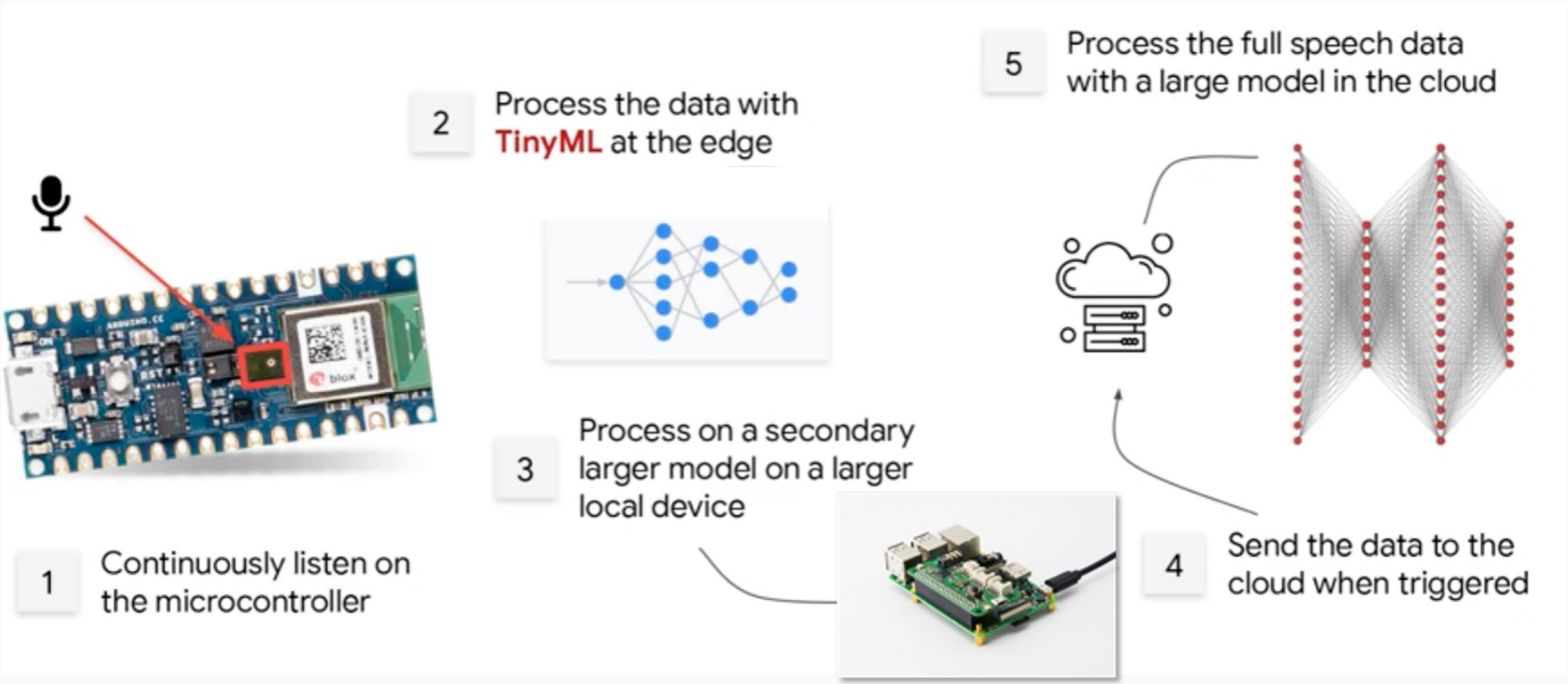
- **General ASR*** still requires **larger, power-hungry models**
  - But it can run on mobile devices (offline dictation on smartphones)

    * Automatic Speech Recognition

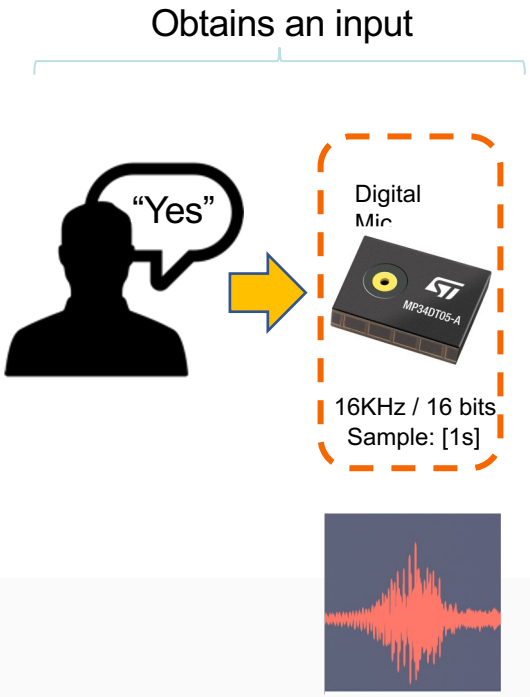
Personal Assistant



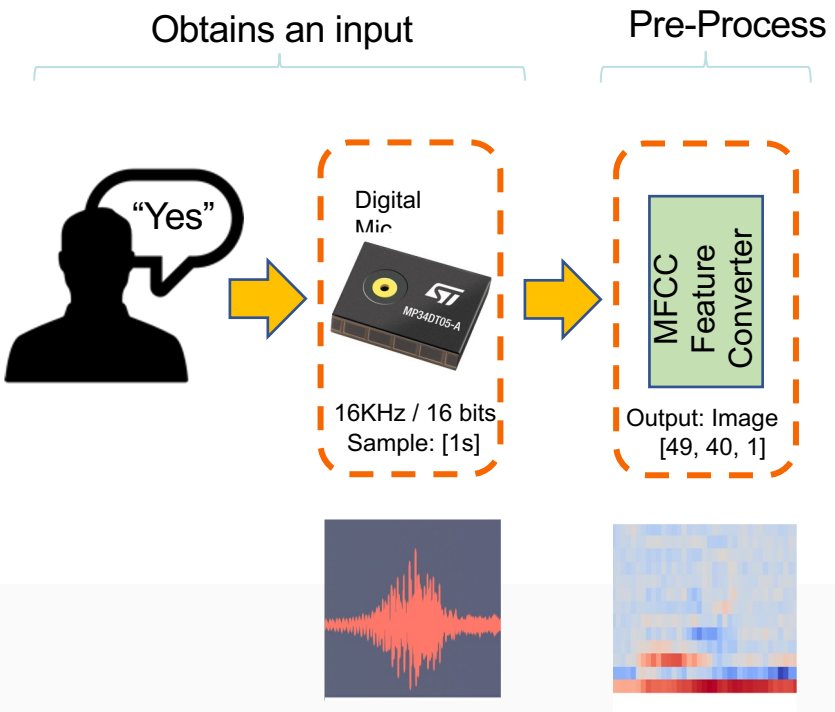
“Cascade” Detection: multi-stage model



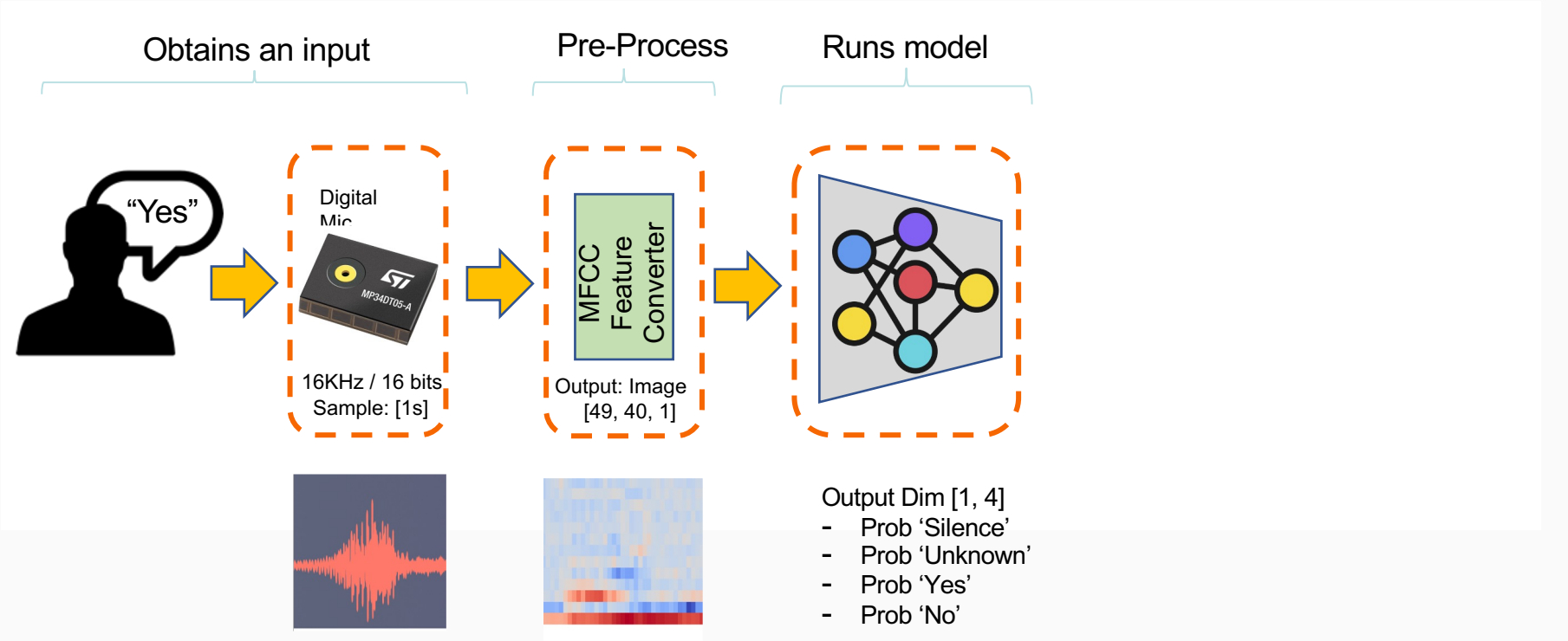
KeyWord Spotting (KWS) - **Inference**



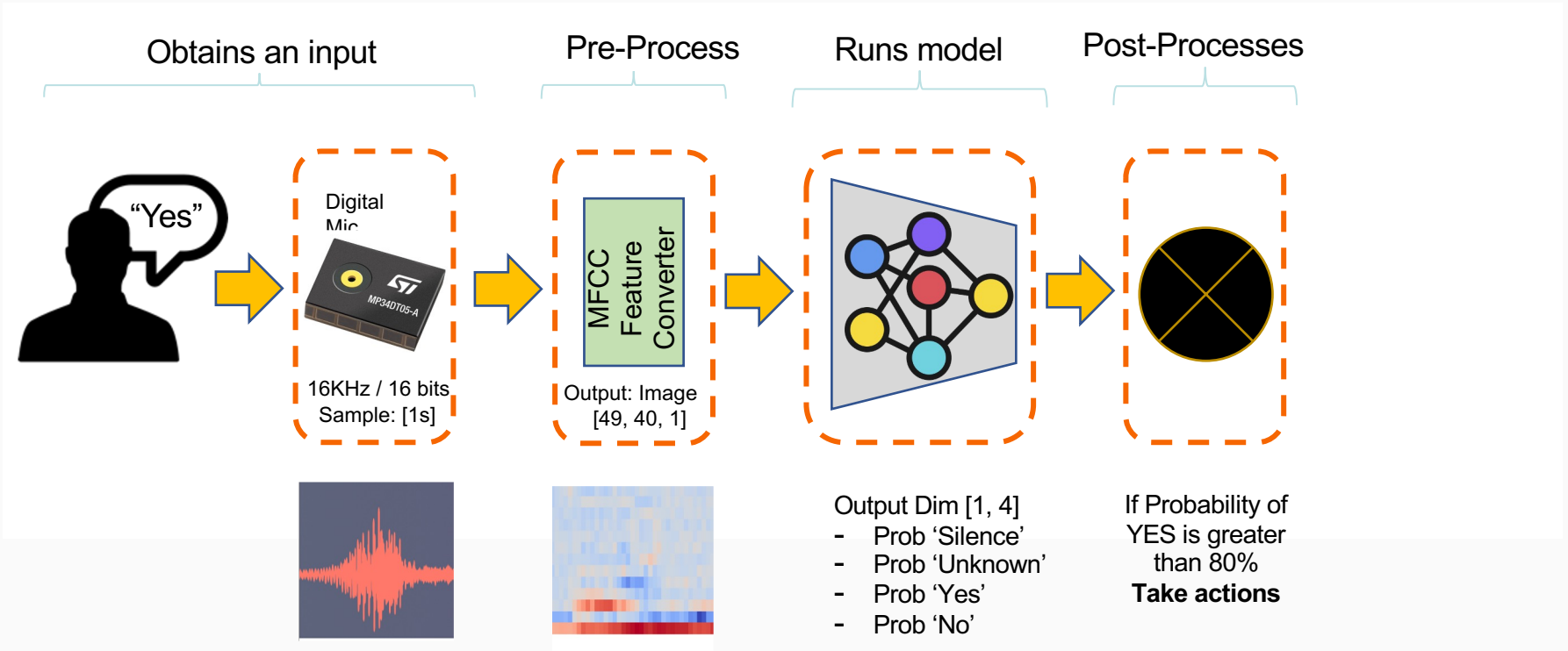
KeyWord Spotting (KWS) - **Inference**



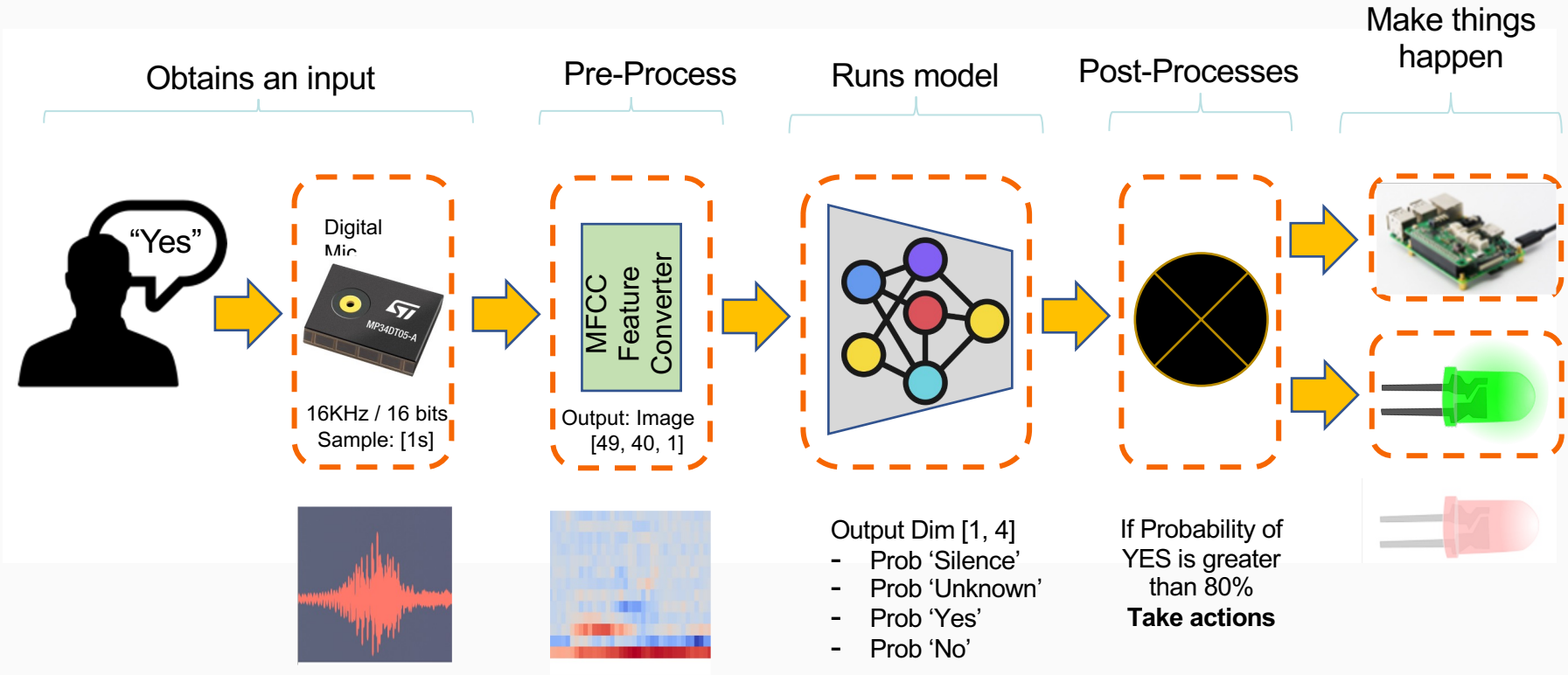
KeyWord Spotting (KWS) - **Inference**



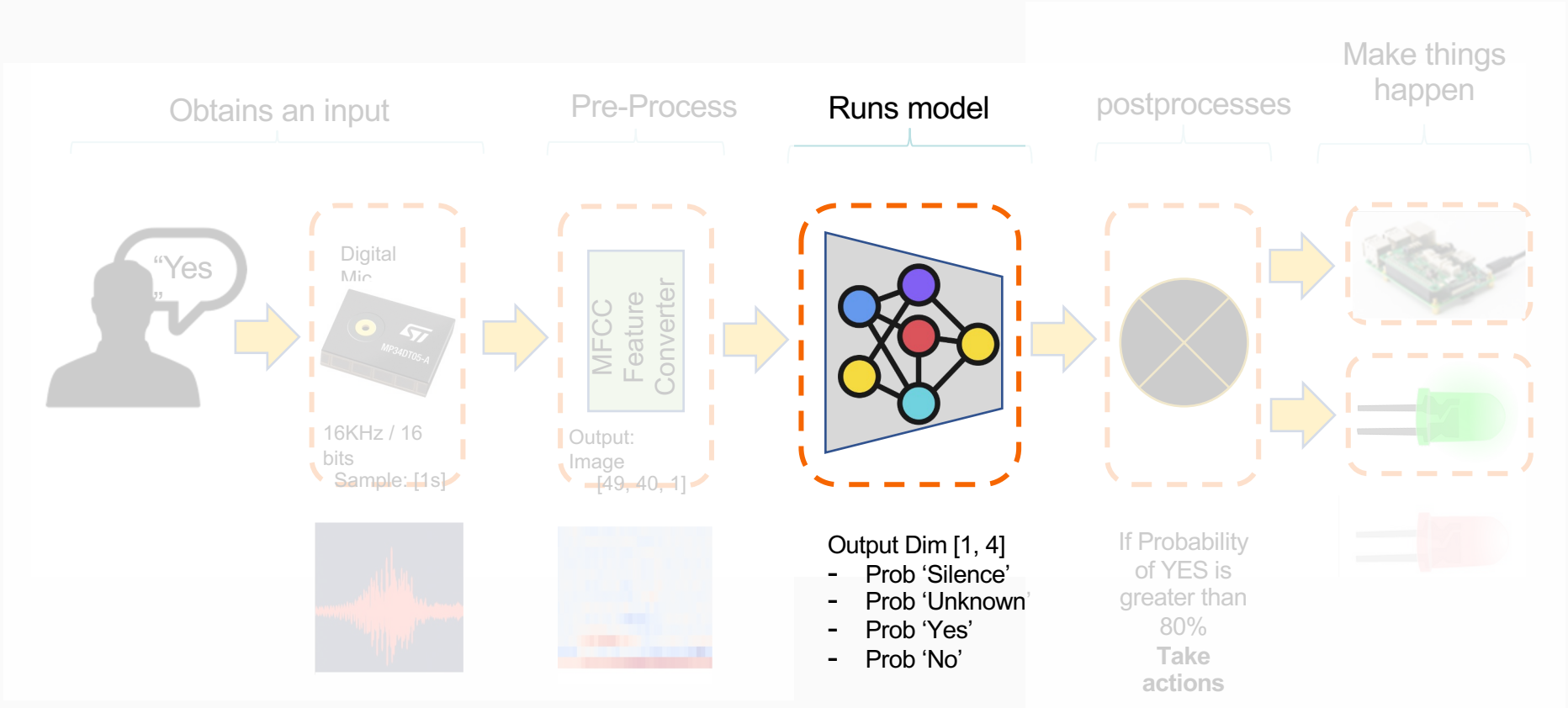
KeyWord Spotting (KWS) - **Inference**



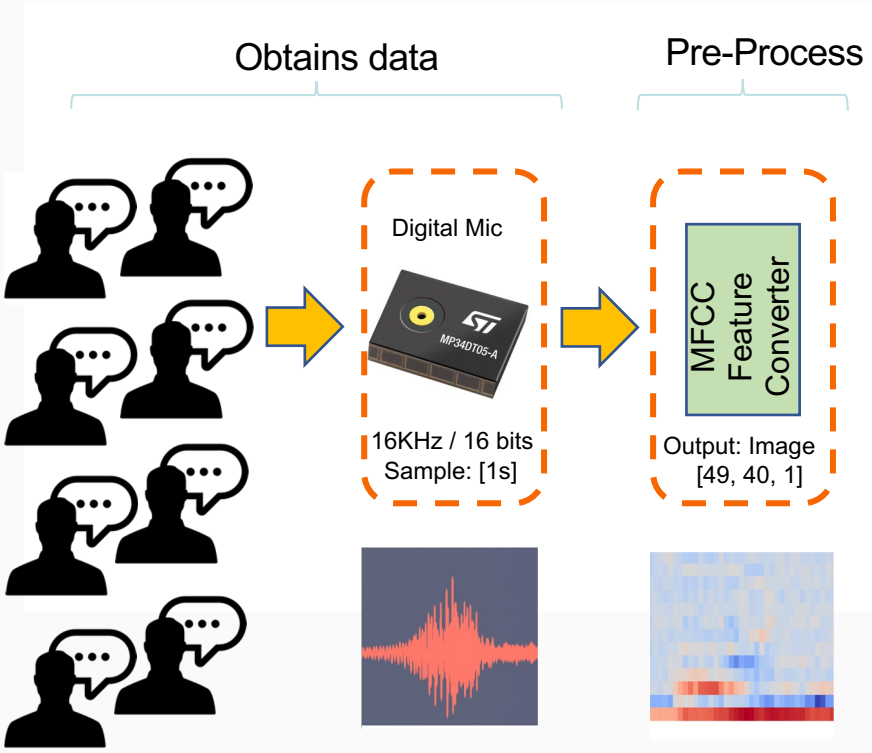
KeyWord Spotting (KWS) - **Inference**



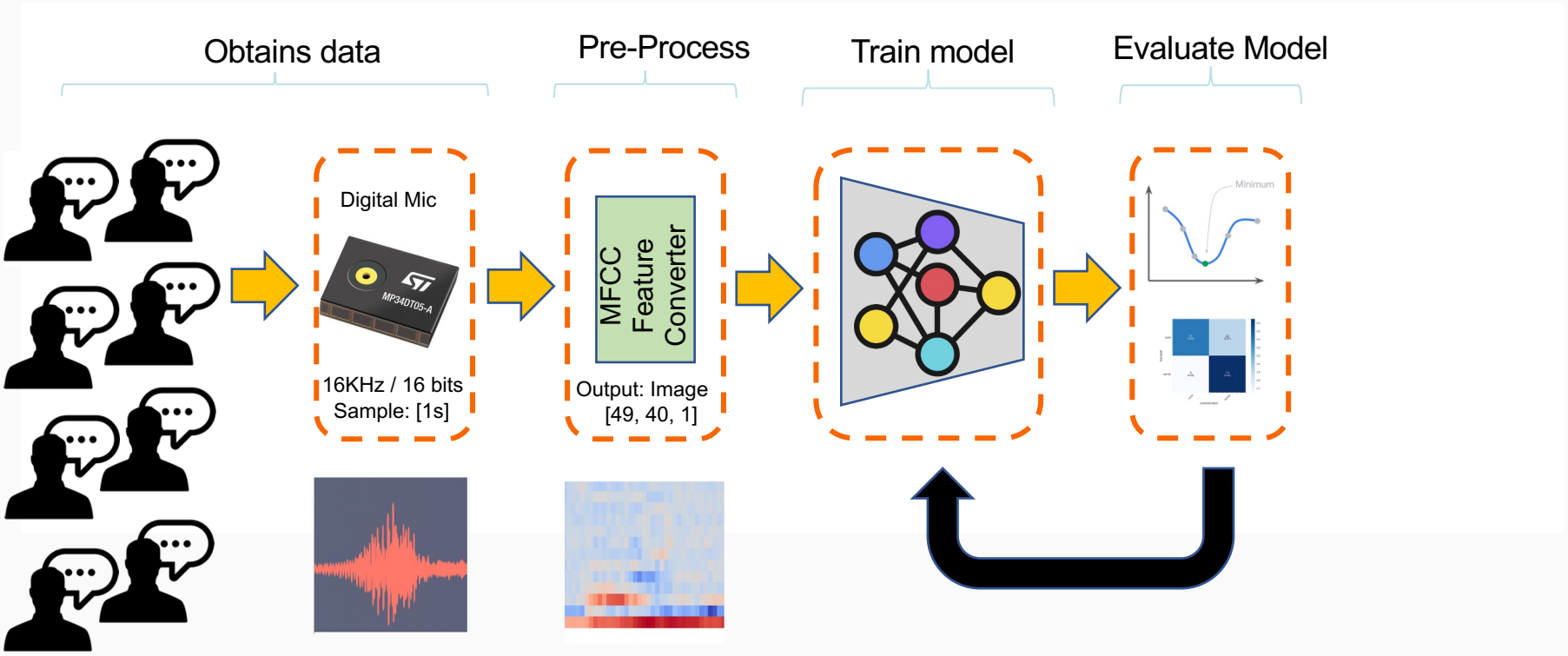
KeyWord Spotting (KWS) - **Model**



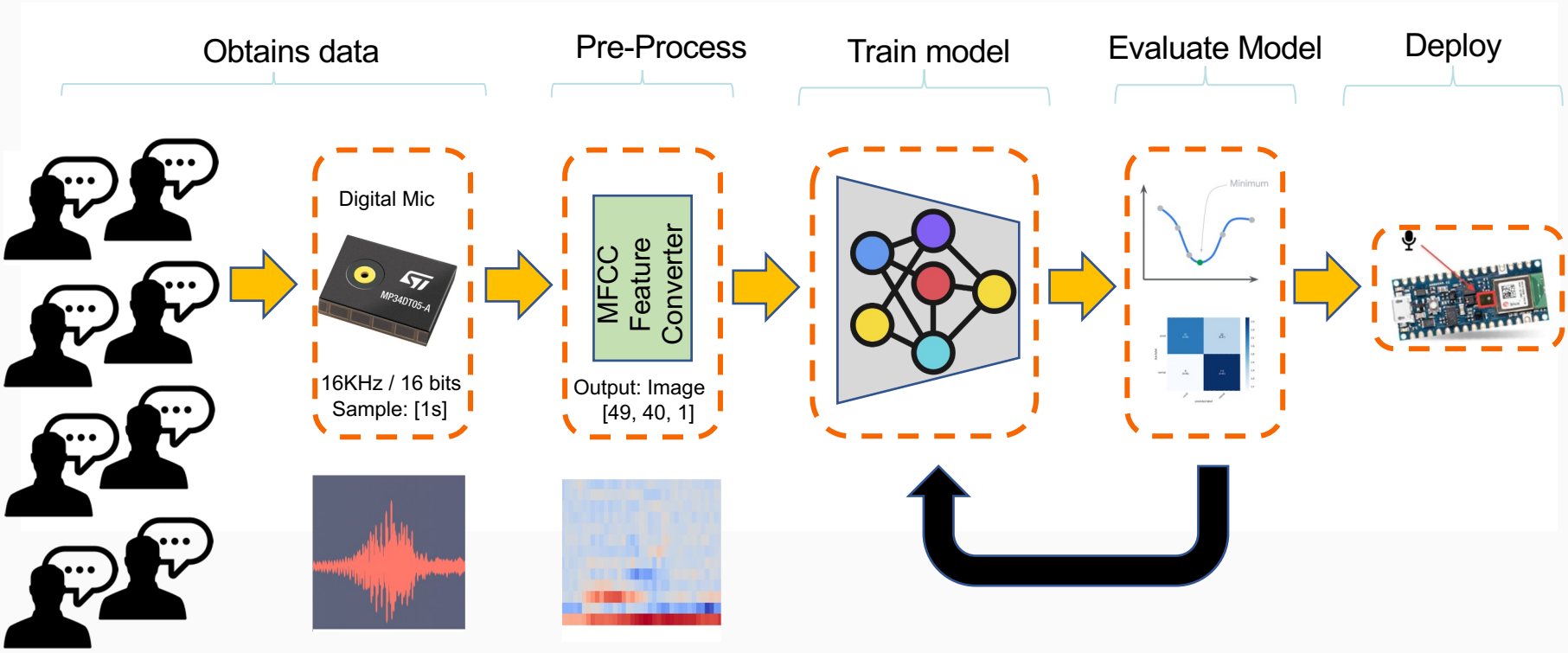
KeyWord Spotting (KWS) – **Create Model (Training)**



KeyWord Spotting (KWS) – **Create Model (Training)**

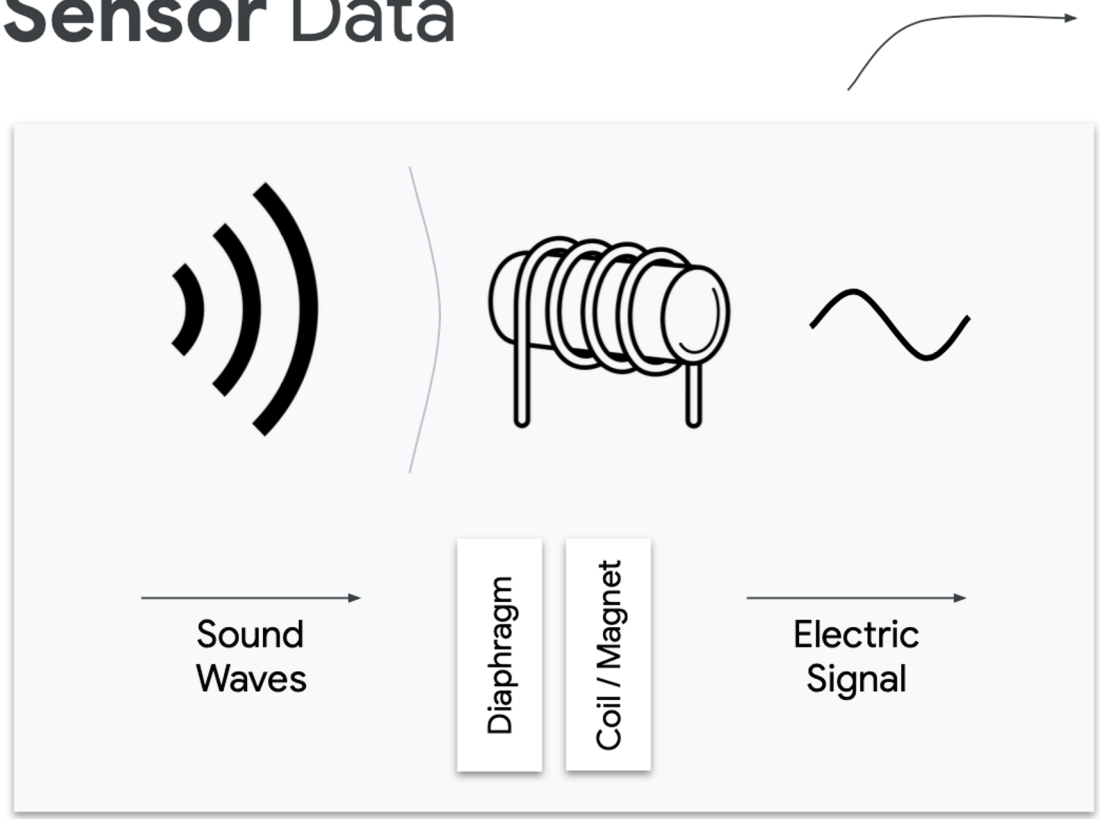


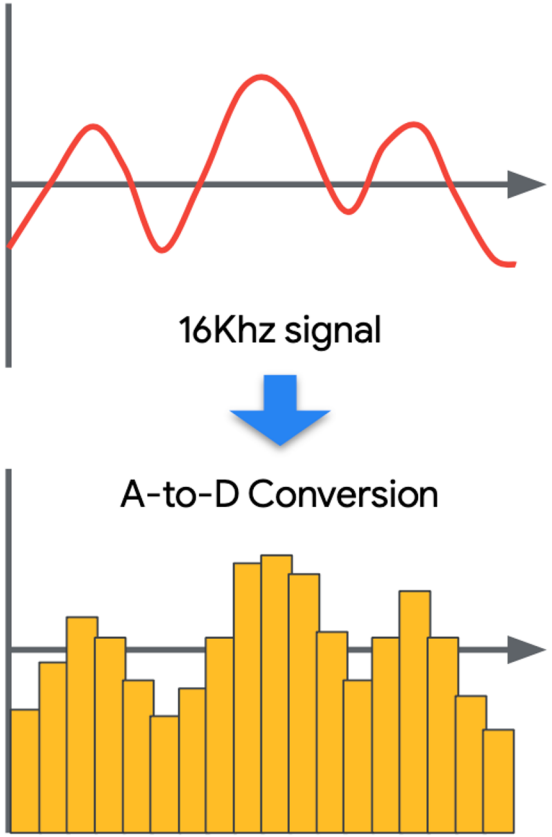
KeyWord Spotting (KWS) – **Create Model (Training)**



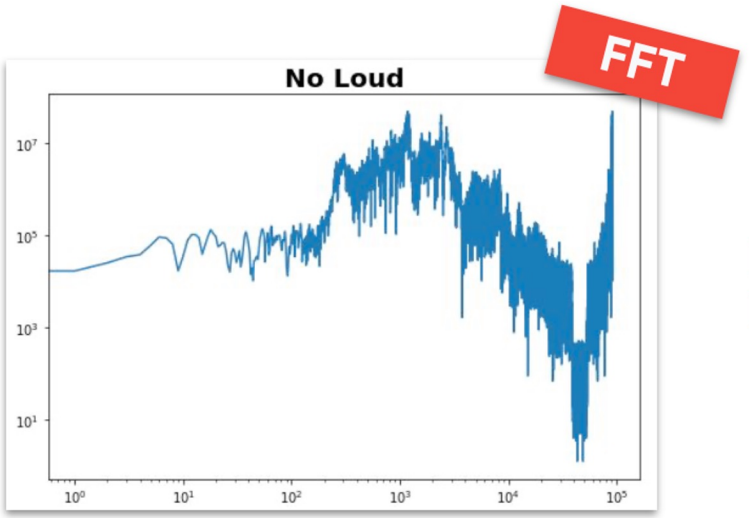
KWS Data Collection & Pre-Processing

Sensor Data

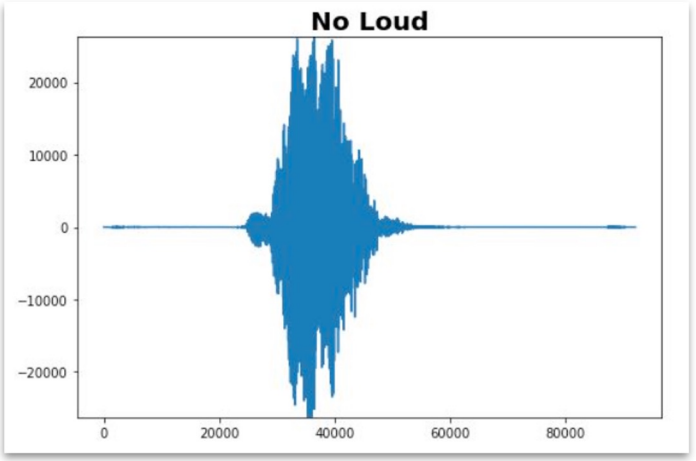

Sound
Waves
Diaphragm
Coil / Magnet
Electric
Signal


16Khz signal
A-to-D Conversion

Signal **Components**?



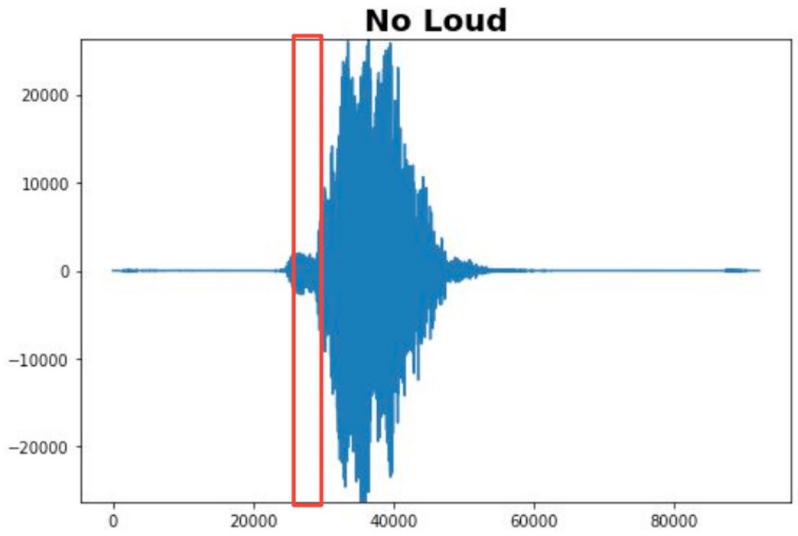




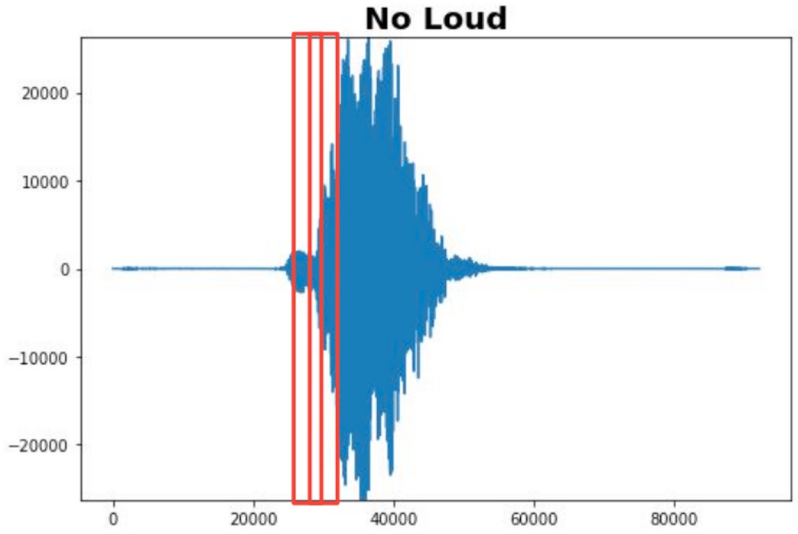
Frequency

Time

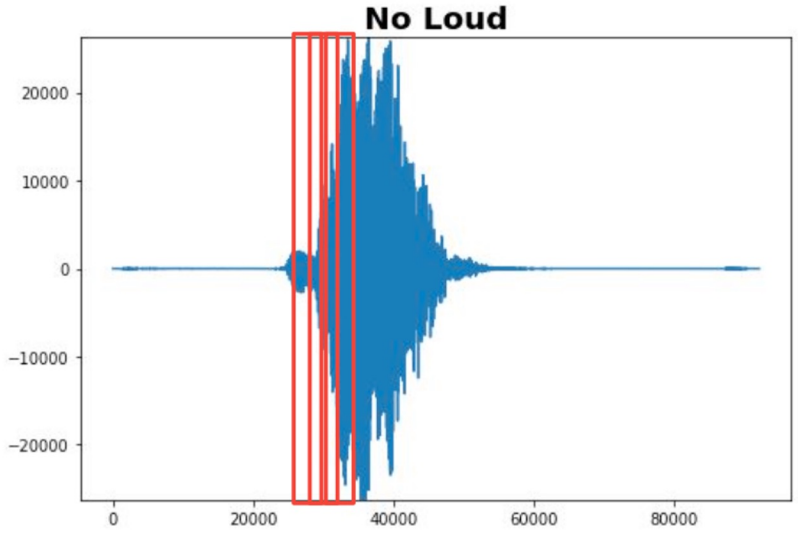
Data Preprocessing



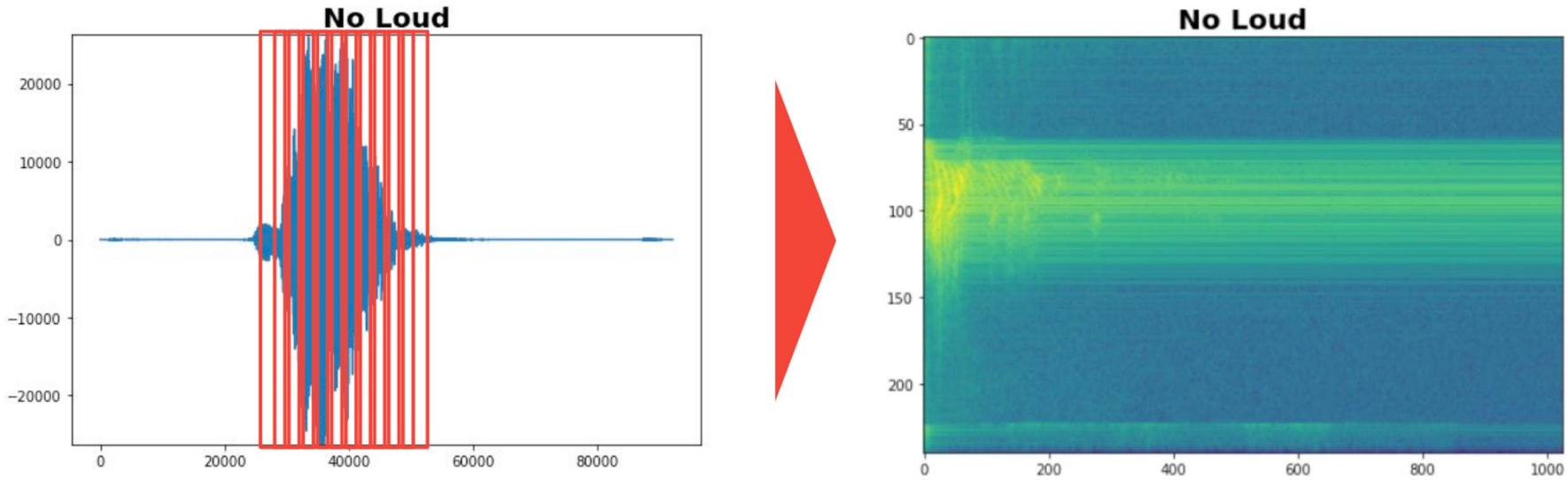
Data Preprocessing



Data Preprocessing

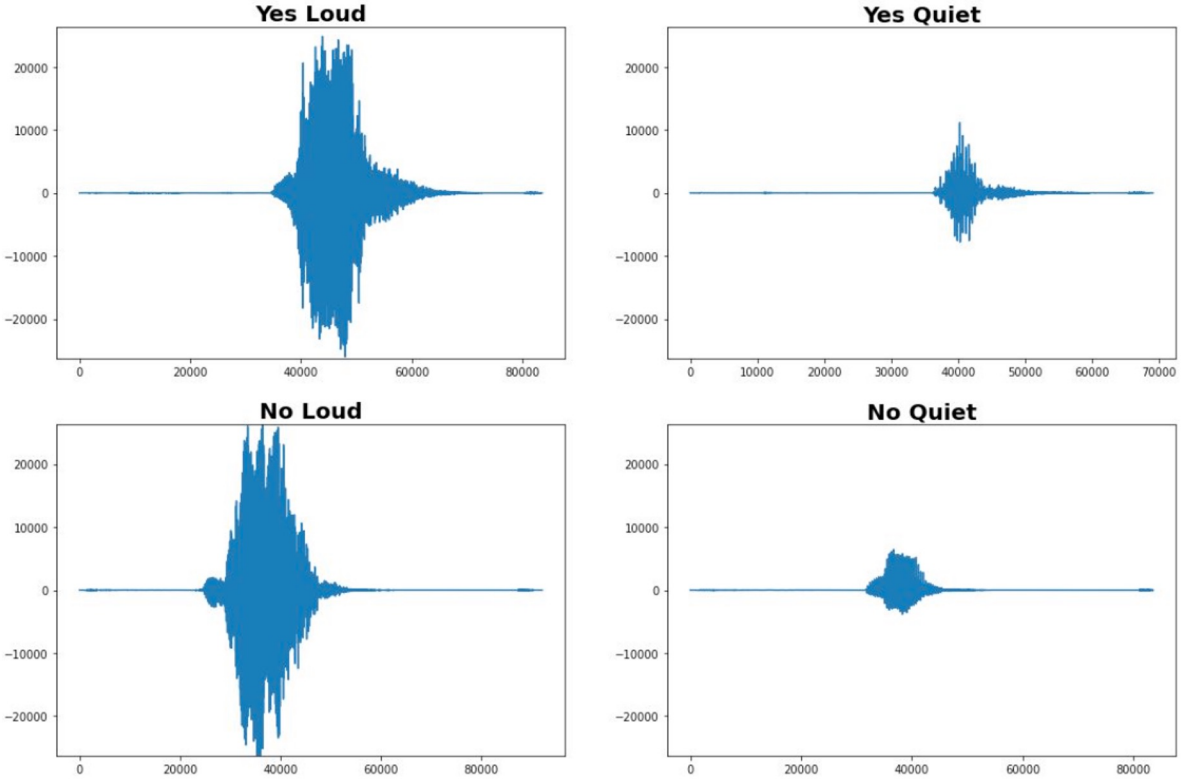


Data Preprocessing: **Spectrograms**

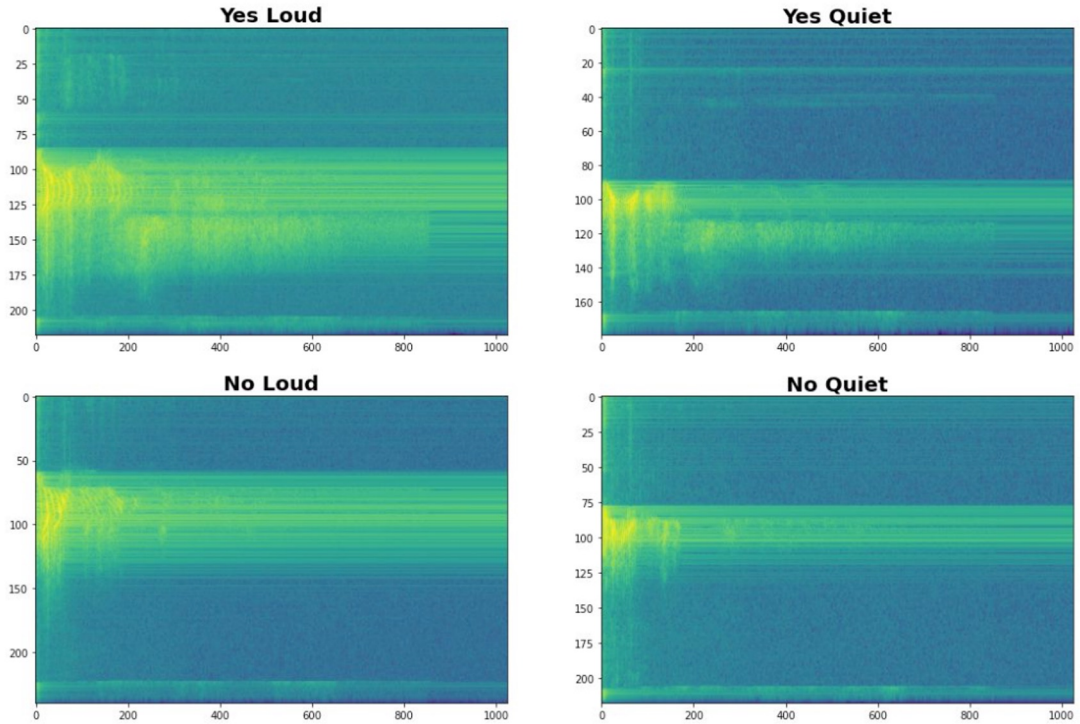


- A spectrogram is a picture of how the frequency content of a signal changes over time. It has three dimensions:
- **x-axis: time**
- **y-axis: frequency**
- **colour or brightness: intensity**, meaning how much energy the signal has at that frequency at that moment, usually on a dB (log) scale.

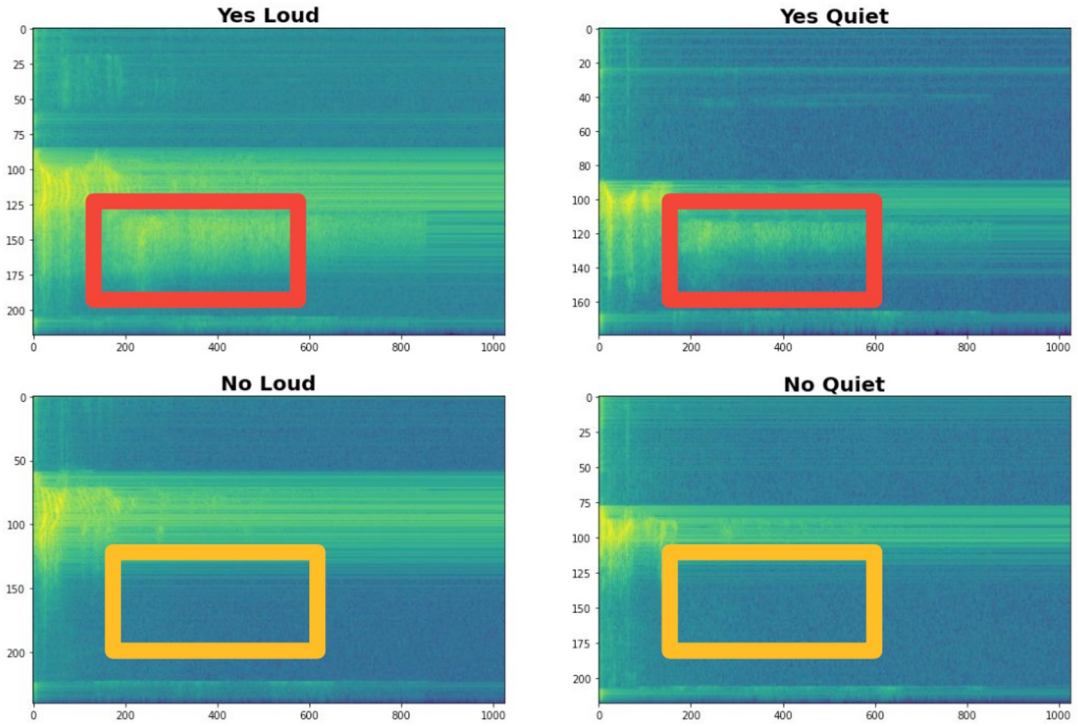
Data Preprocessing: **Spectrograms**

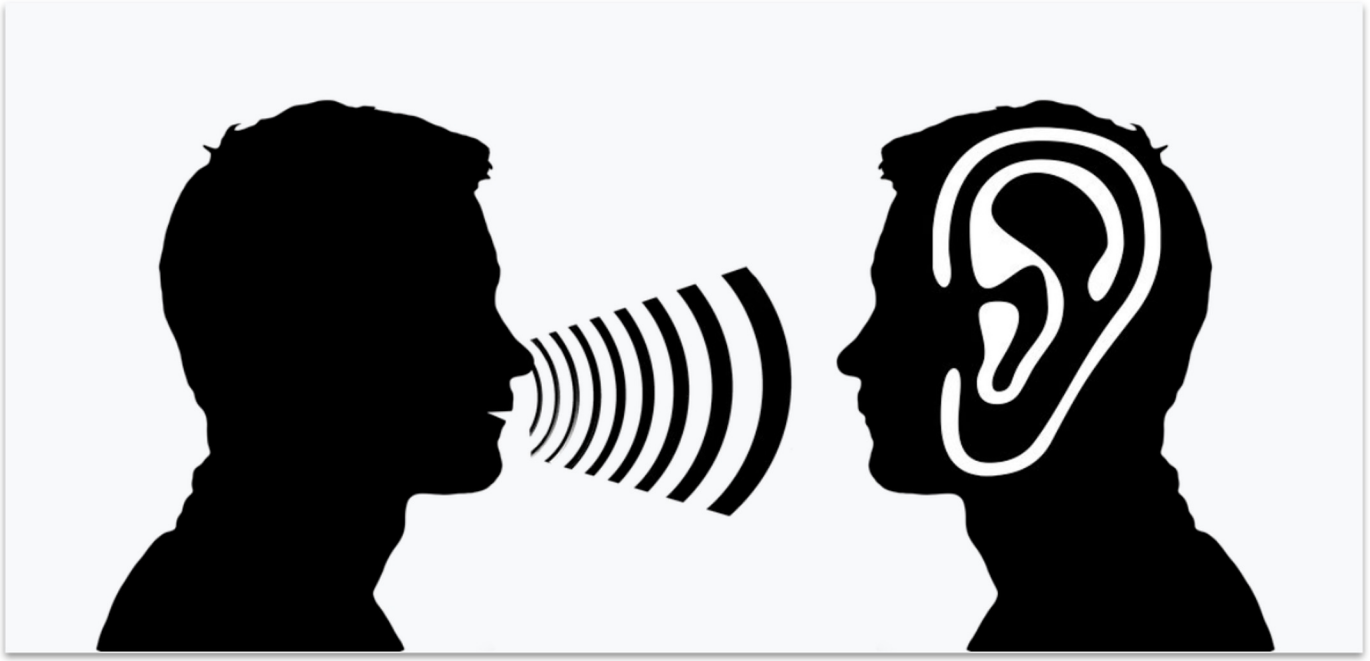


Data Preprocessing: **Spectrograms**



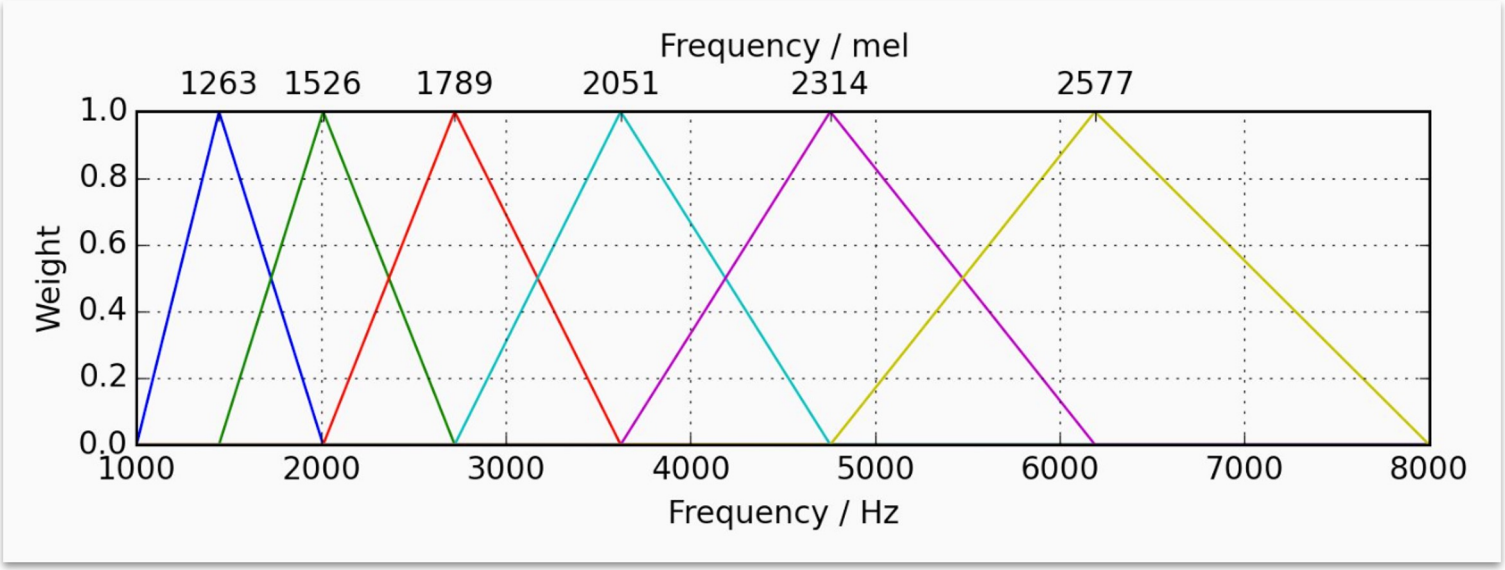
Data Preprocessing: **Spectrograms**



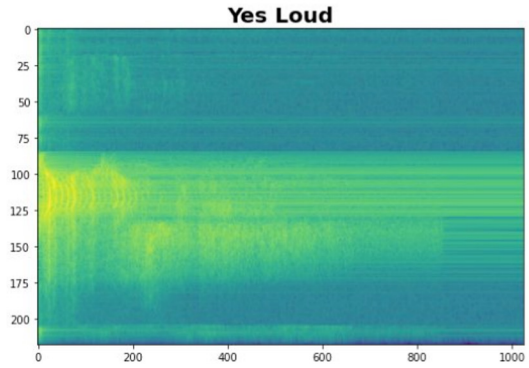


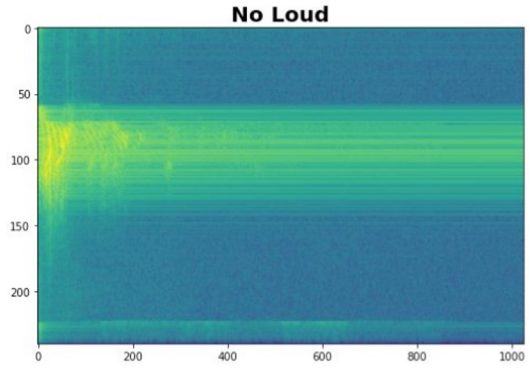
The lower band frequencies is much more crisper to us

Mel Filterbanks

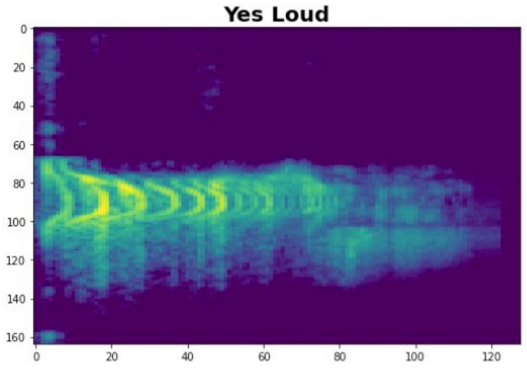


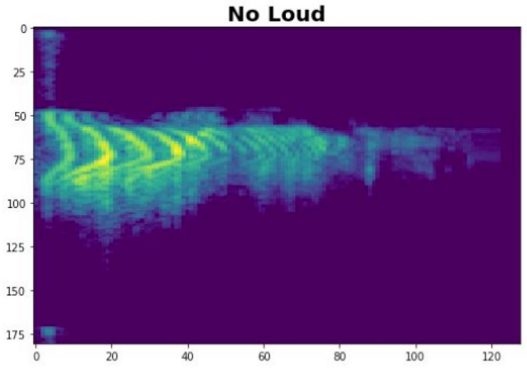
Spectrograms v. MFCCs



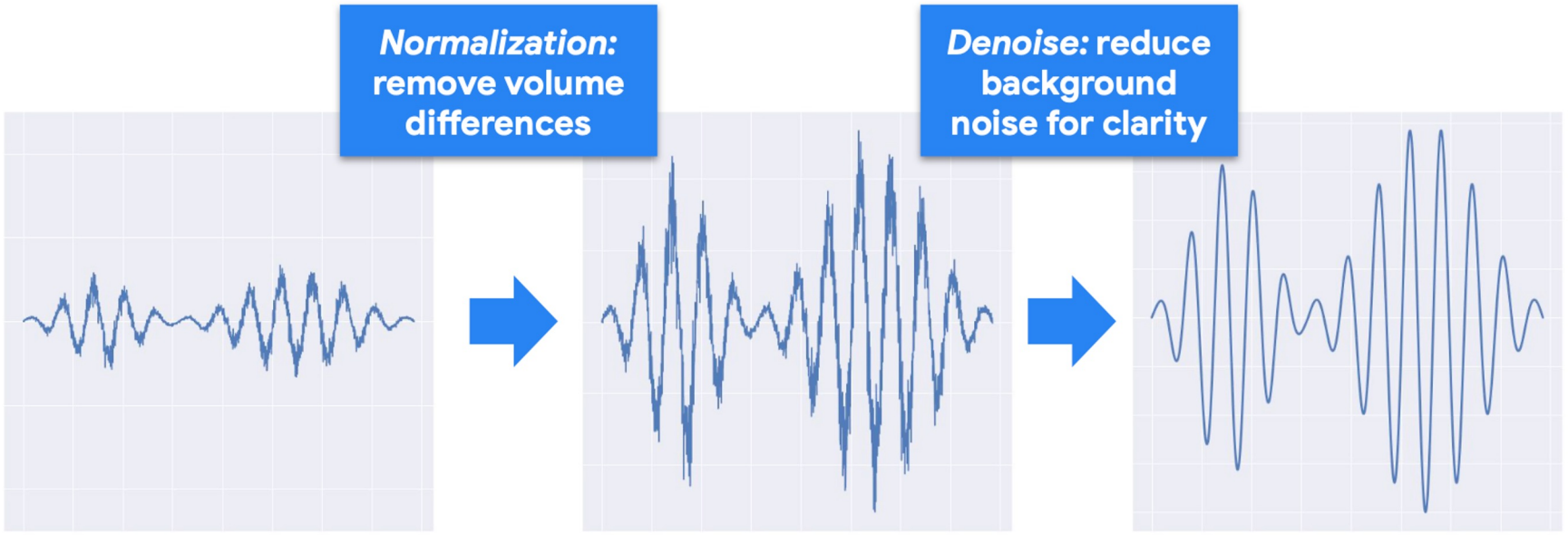








Additional **Feature Engineering**



EdgeImpulse

Lab

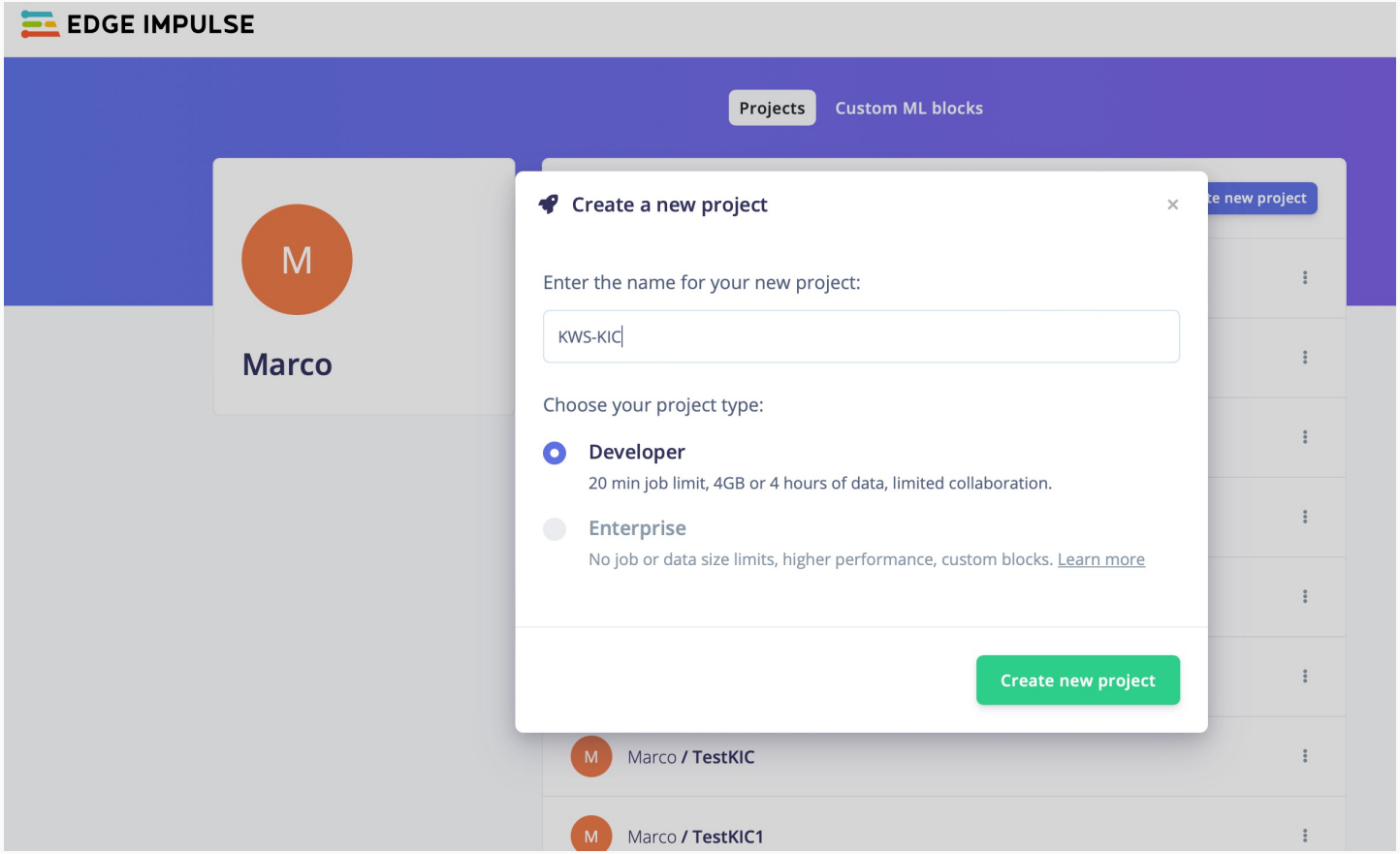
Goal:

- develop a system that can blink an LED when it detects the word Yes (or Hai or Naeam or Si or Oui or…)



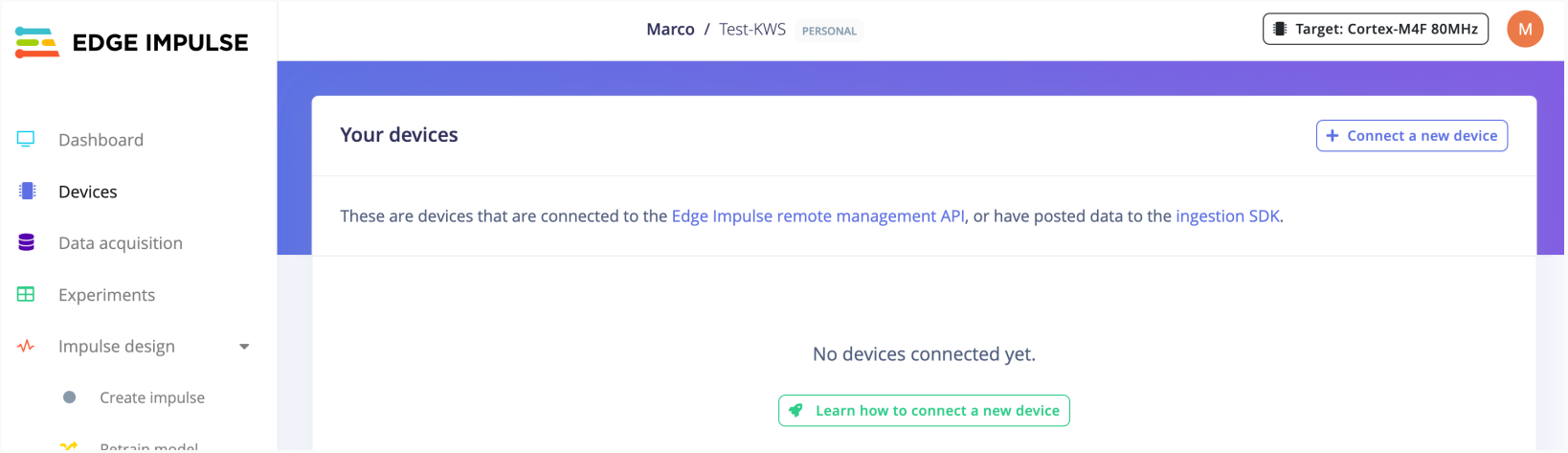
Lab

Login into EdgeImpulse and create a new project.

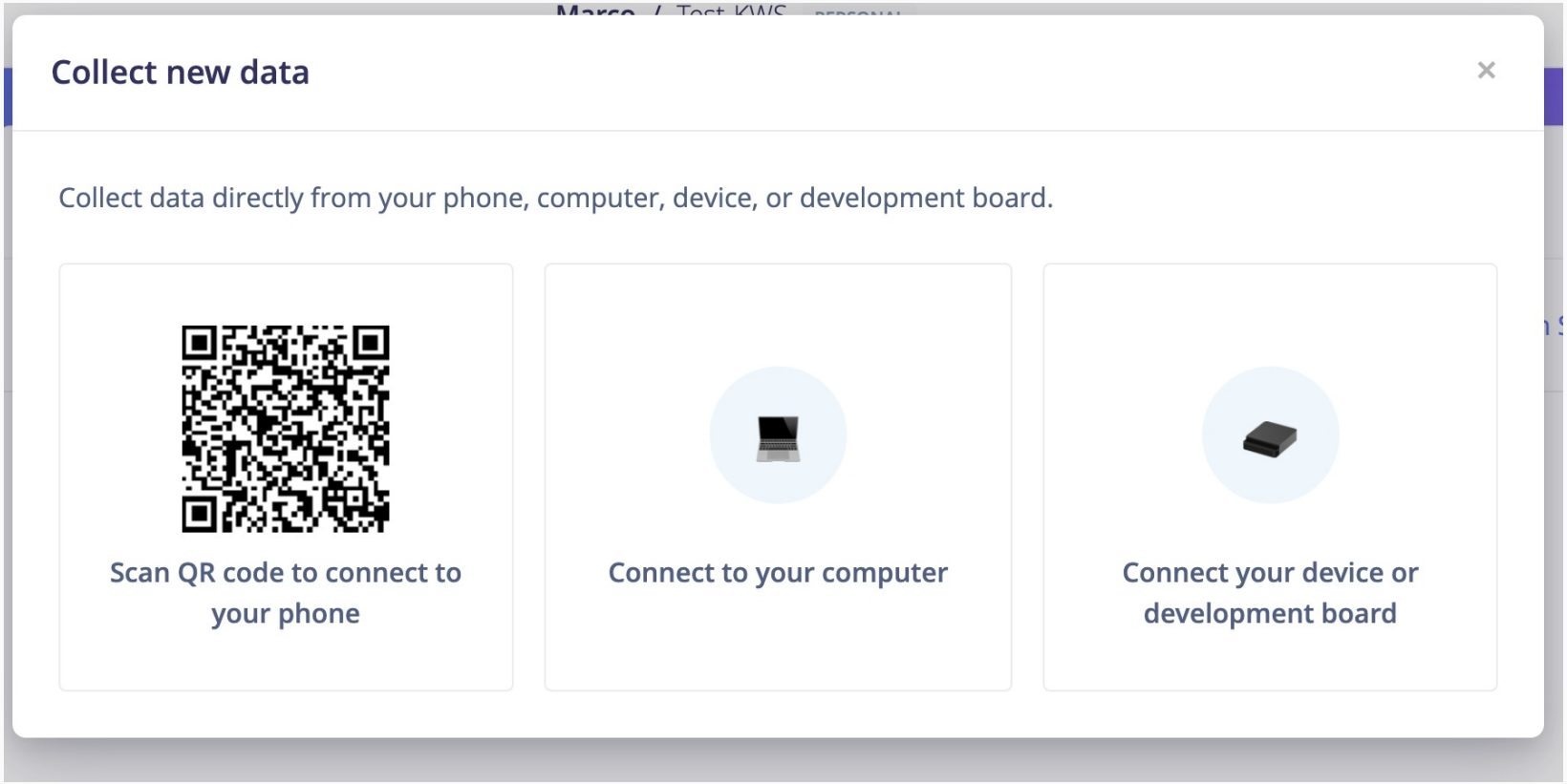


Lab

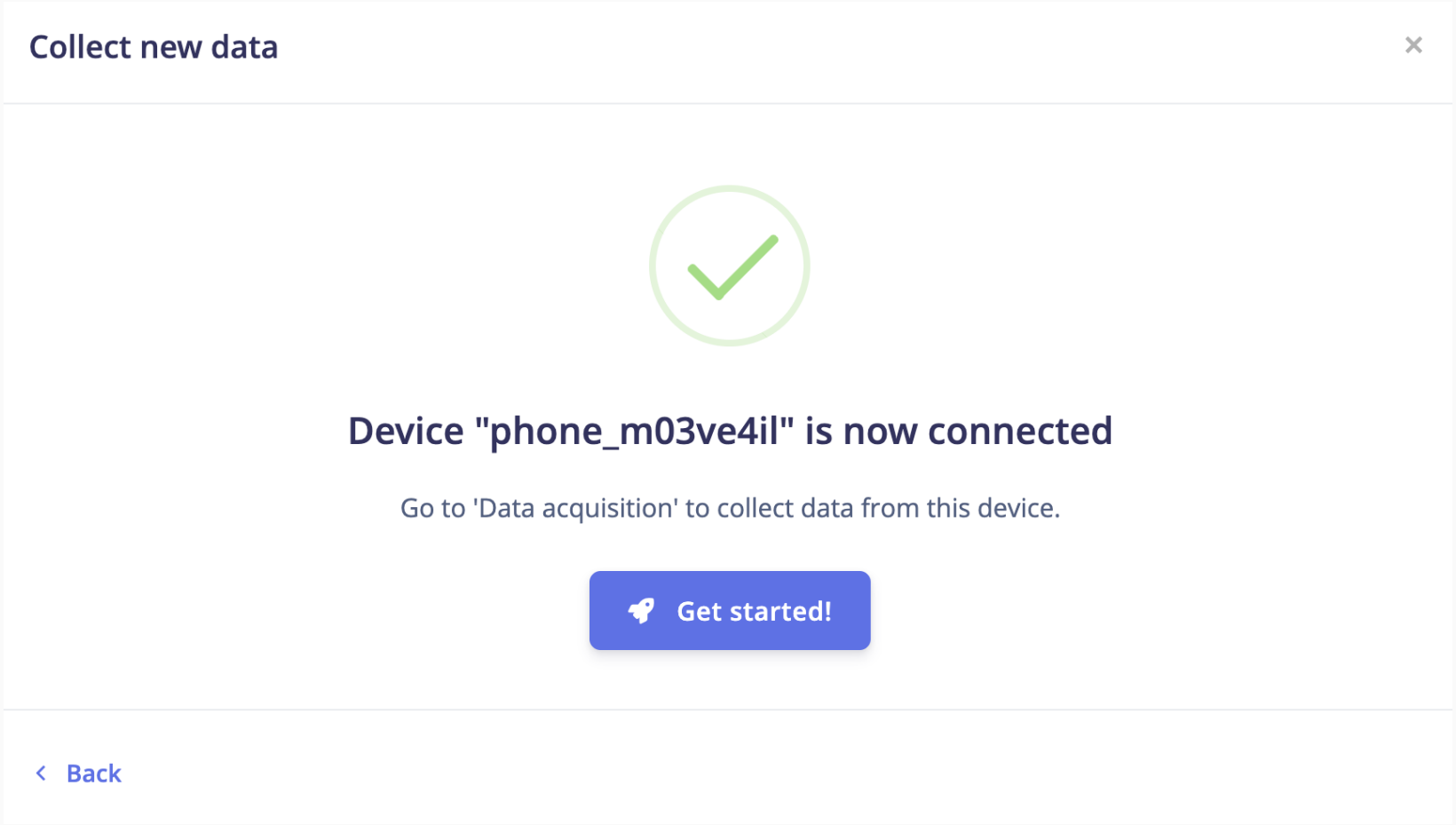
Once the project is created, click on **Devices** on the left to add your smartphone as input device. You could also upload data from an existing dataset or add a sensor.



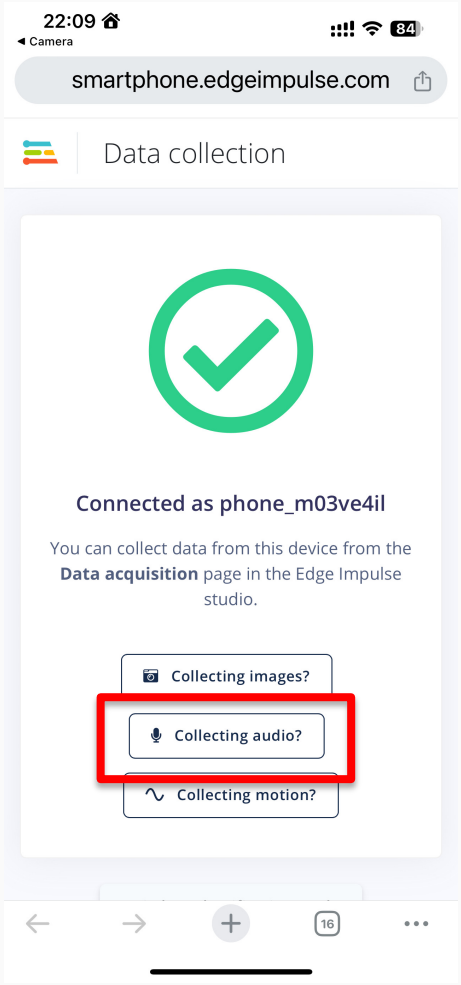
Lab


Collect new data
Collect data directly from your phone, computer, device, or development board.
Scan QR code to connect to your phone
Connect to your computer
Connect your device or development board

Lab


Collect new data
Device "phone_m03ve4il" is now connected
Go to 'Data acquisition' to collect data from this device.
Get started!
Back

Lab


22:09
Camera
84
smartphone.edgeimpulse.com
Data collection
Connected as phone_m03ve4il
You can collect data from this device from the **Data acquisition** page in the Edge Impulse studio.
Collecting images?
Collecting audio?
Collecting motion?
16

Split data

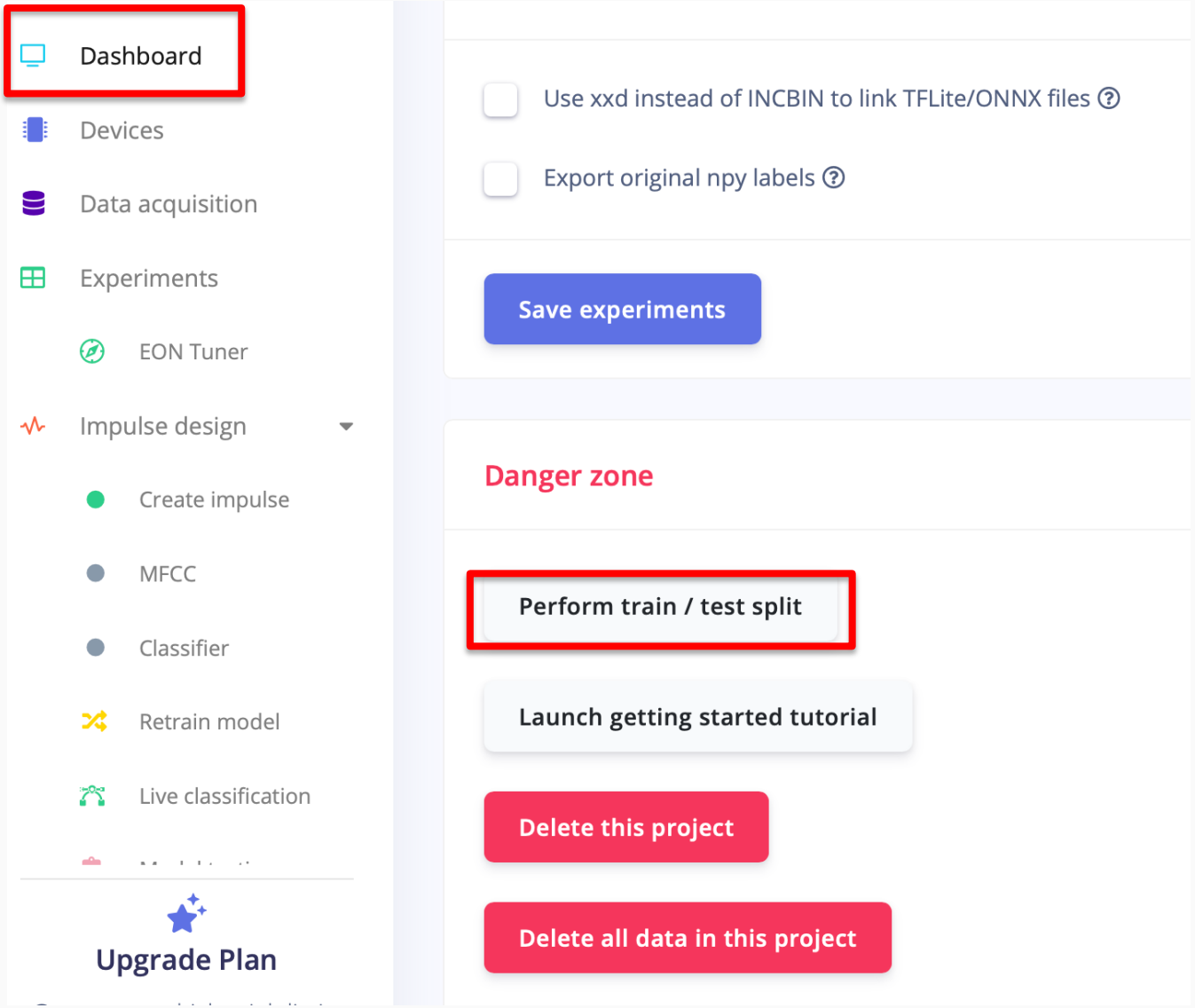
To detect a machine learning model behavior, we need to use observations that aren't used in the training process. Otherwise, the evaluation of the model would be biased.

Using the training observations for model evaluation is like giving a class of students a set of questions, then asking some of the questions in the final exam. We can't know whether the students really learned the subject or just memorized some specific data.

The simplest method is to divide the whole dataset into two sets. Then use one for training and the other for model evaluation.

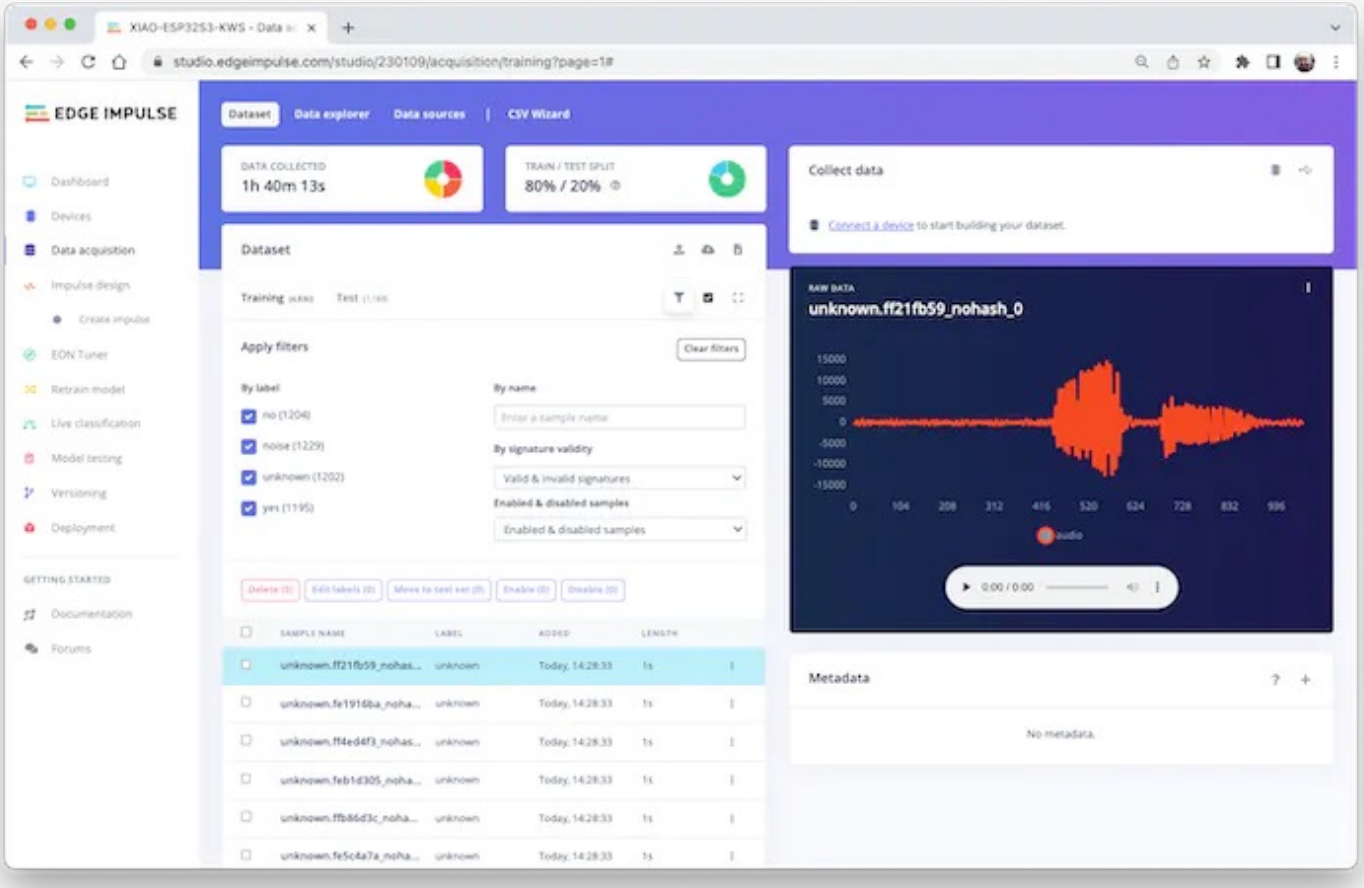
Empirical studies show that the best results are obtained if we use **20-30% of the data for testing**, and the remaining **70-80% of the data for training.**

Split data


Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
MFCC
Classifier
Retrain model
Live classification
Upgrade Plan
Use xxd instead of INCBIN to link TFLite/ONNX files
Export original npy labels
Save experiments
Danger zone
Perform train / test split
Launch getting started tutorial
Delete this project
Delete all data in this project

Lab

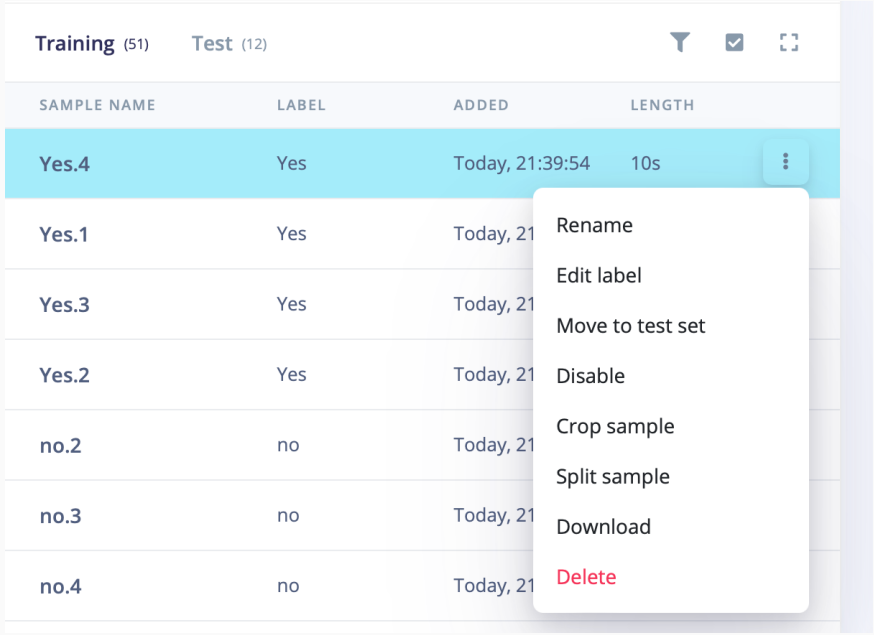
The samples you have collected with the phone will now appear in the Data acquisition section.



Lab

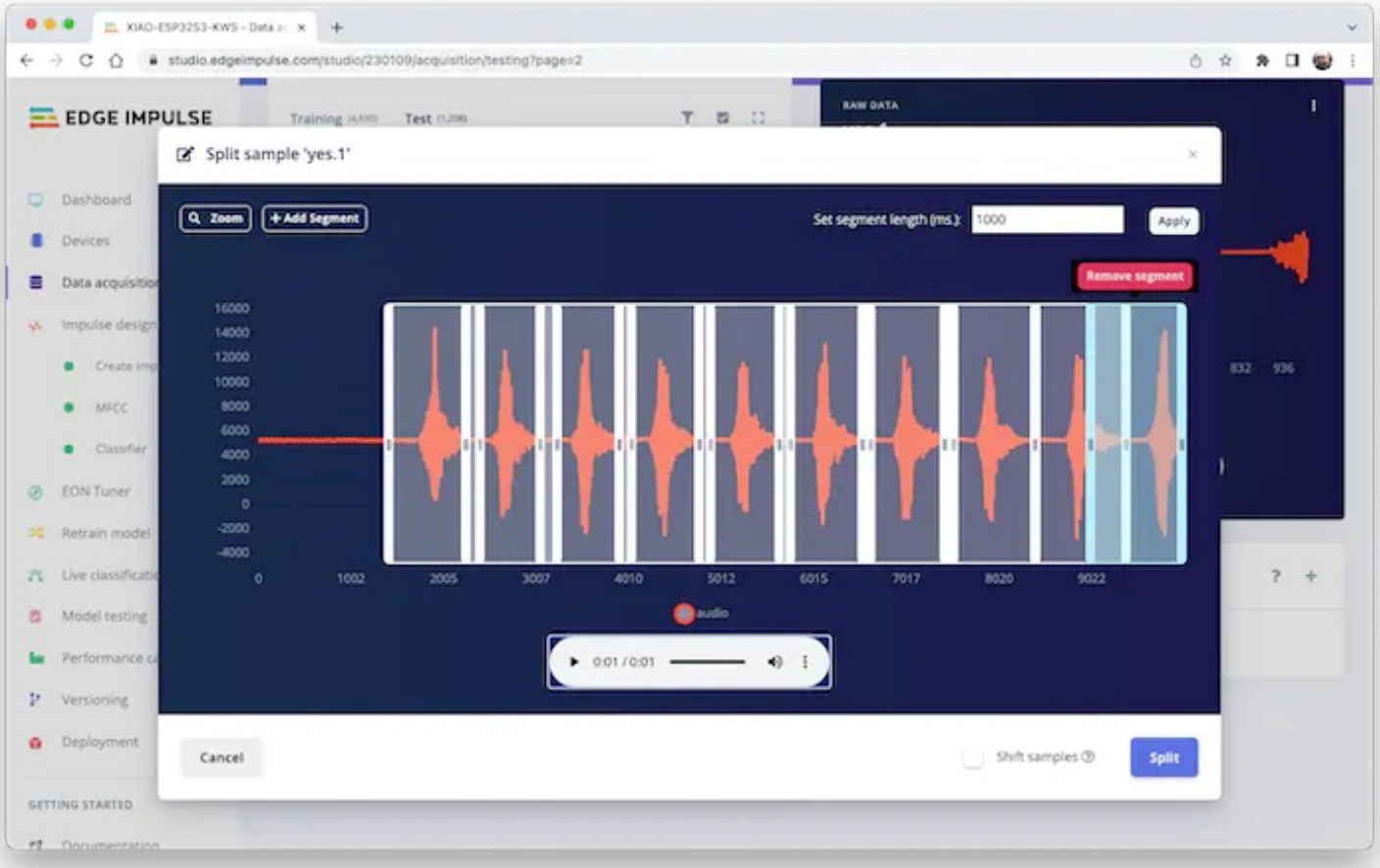
All data from the Edge Impulse dataset have a 1 second length. We might want to split the 1 second samples into smaller chunks.

Click on three dots after the sample name and select **Split sample**.



Lab

Once inside the tool, split the dat. If necessary, add or remove segments:



Lab

This procedure should be repeated for all samples.

Note: For longer audio files (minutes), first, split into 10-second segments and after that, use the tool again to get the final 1-second splits.

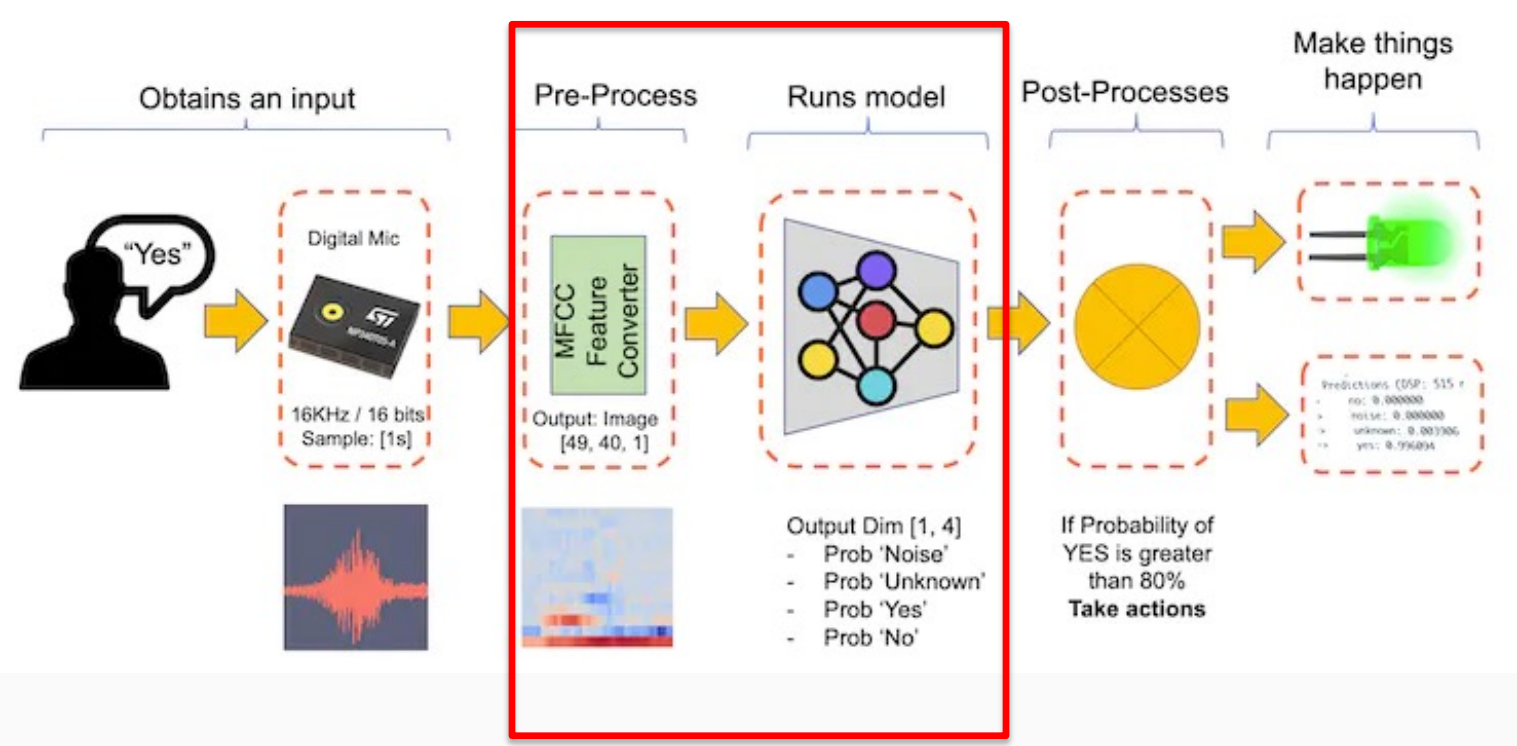


It is important for the dataset to be balanced!



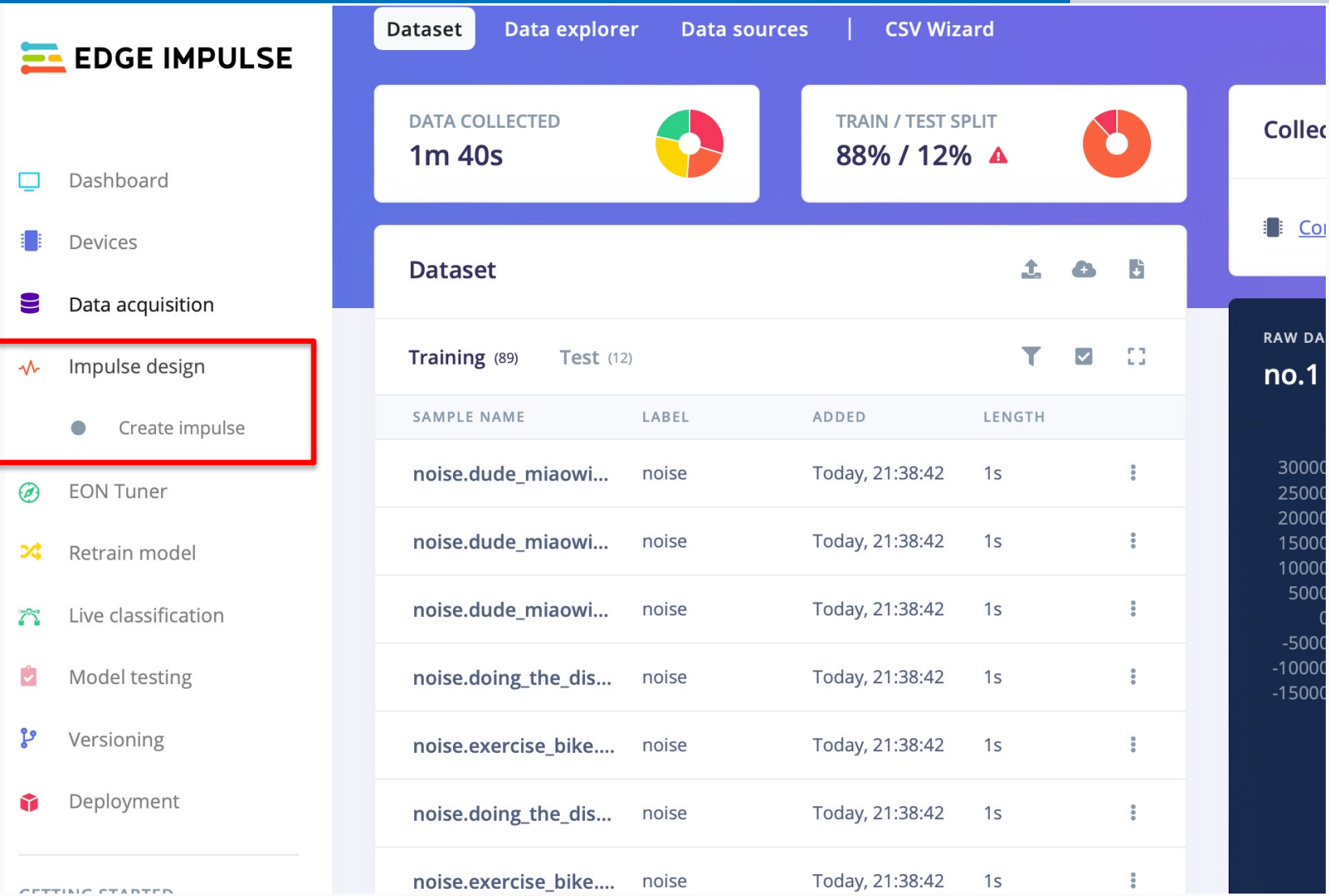
Lab

We will now be creating an Impulse (Pre-Process / Model definition)



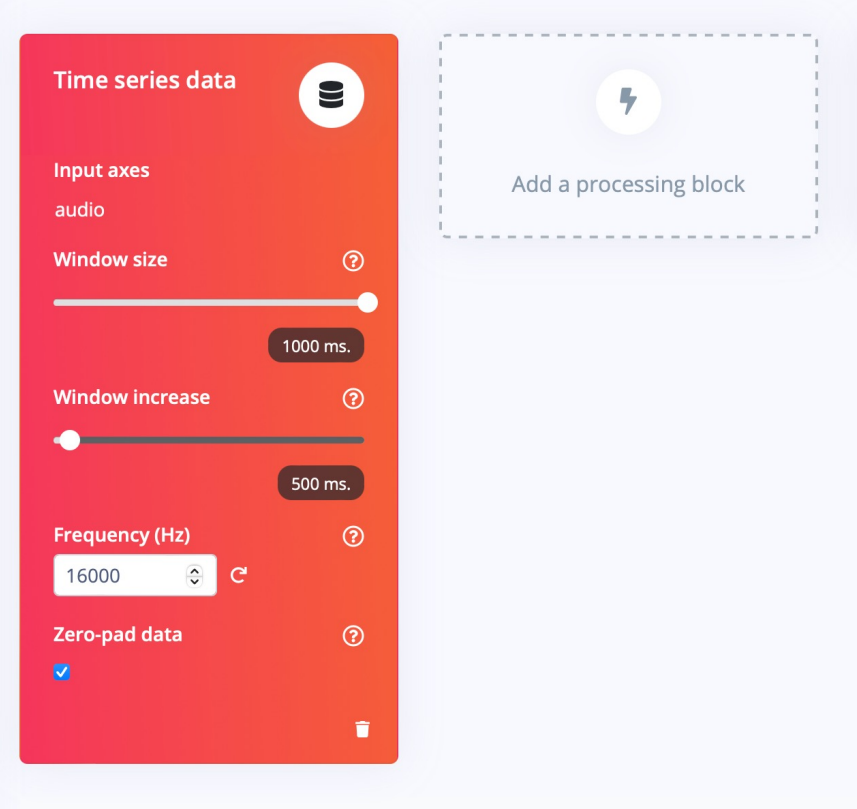
An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data.

Lab


EDGE IMPULSE
Dashboard
Devices
Data acquisition
Impulse design
Create impulse
EON Tuner
Retrain model
Live classification
Model testing
Versioning
Deployment
Dataset
Data explorer
Data sources
CSV Wizard
DATA COLLECTED
1m 40s
TRAIN / TEST SPLIT
88% / 12%
Dataset
Training (89)
Test (12)
SAMPLE NAME
LABEL
ADDED
LENGTH
noise.dude_miaowi...
noise
Today, 21:38:42
1s
noise.dude_miaowi...
noise
Today, 21:38:42
1s
noise.dude_miaowi...
noise
Today, 21:38:42
1s
noise.doing_the_dis...
noise
Today, 21:38:42
1s
noise.exercise_bike....
noise
Today, 21:38:42
1s
noise.doing_the_dis...
noise
Today, 21:38:42
1s
noise.exercise_bike....
noise
Today, 21:38:42
1s
RAW DA
no.1
30000
25000
20000
15000
10000
5000
0
-5000
-10000
-15000

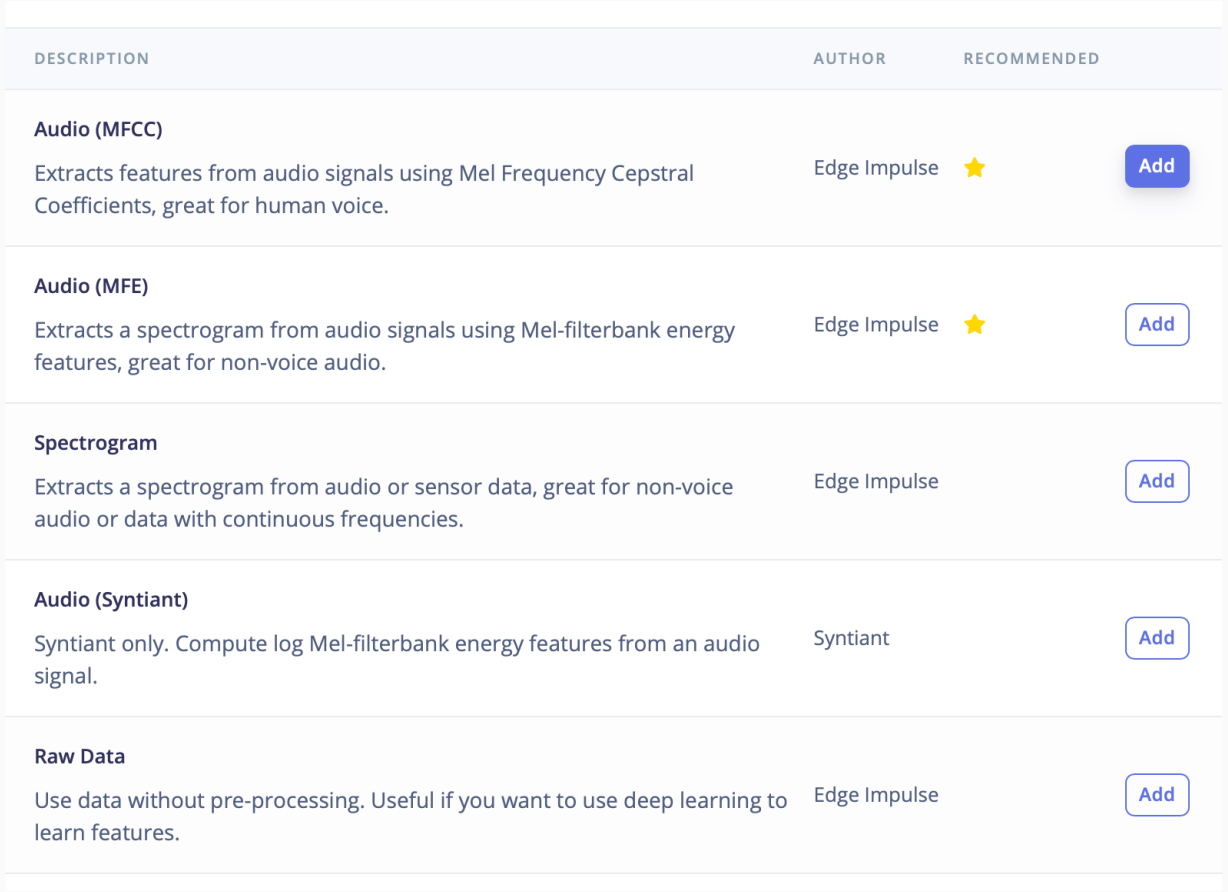
Lab

First, we will take the data points with a 1-second window, augmenting the data, sliding that window each 500ms. Note that the option zero-pad data is set. It is important to fill with zeros samples smaller than 1 second (in some cases, you can reduce the 1000 ms window on the split tool to avoid noises and spikes).



Lab

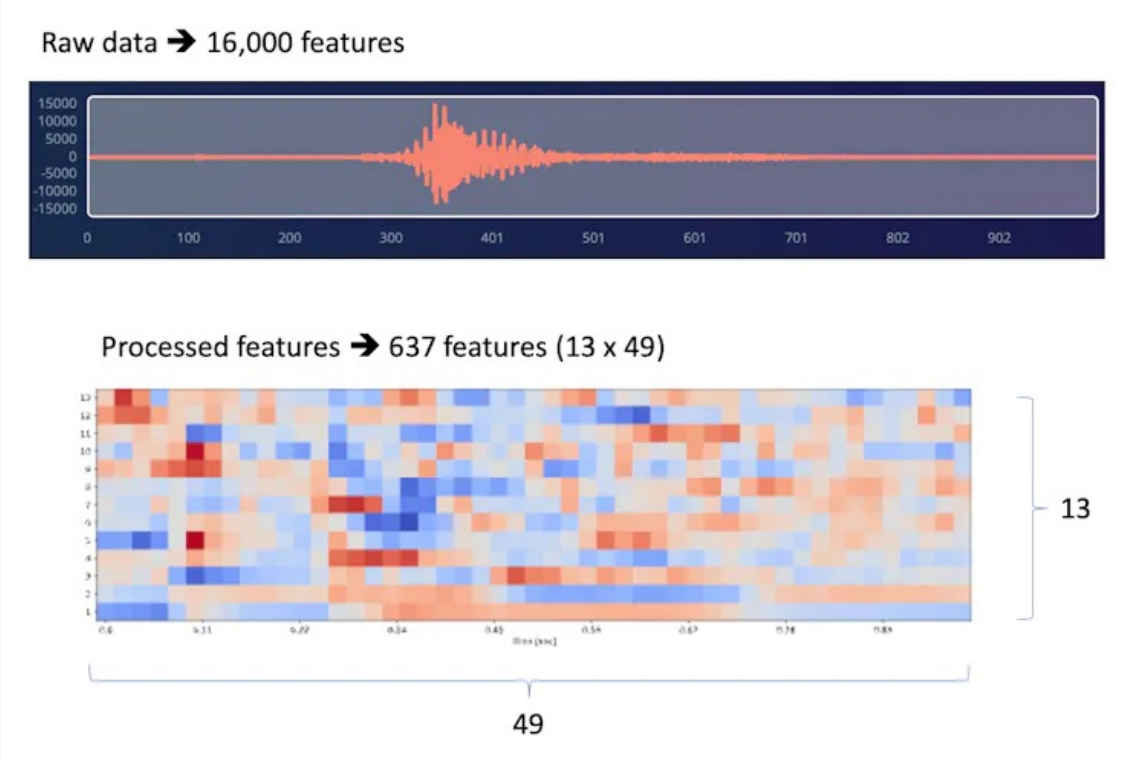
In the Add a processing block, select MFCC.



DESCRIPTION	AUTHOR	RECOMMENDED	
Audio (MFCC) Extracts features from audio signals using Mel Frequency Cepstral Coefficients, great for human voice.	Edge Impulse	★	Add
Audio (MFE) Extracts a spectrogram from audio signals using Mel-filterbank energy features, great for non-voice audio.	Edge Impulse	★	Add
Spectrogram Extracts a spectrogram from audio or sensor data, great for non-voice audio or data with continuous frequencies.	Edge Impulse		Add
Audio (Syntiant) Syntiant only. Compute log Mel-filterbank energy features from an audio signal.	Syntiant		Add
Raw Data Use data without pre-processing. Useful if you want to use deep learning to learn features.	Edge Impulse		Add

Lab

Each 1-second audio sample will be pre-processed and converted to an image (for example, 13 x 49 x 1). MFCC extracts features from audio signals using Mel Frequency Cepstral Coefficients, which are great for the human voice.



MFCC

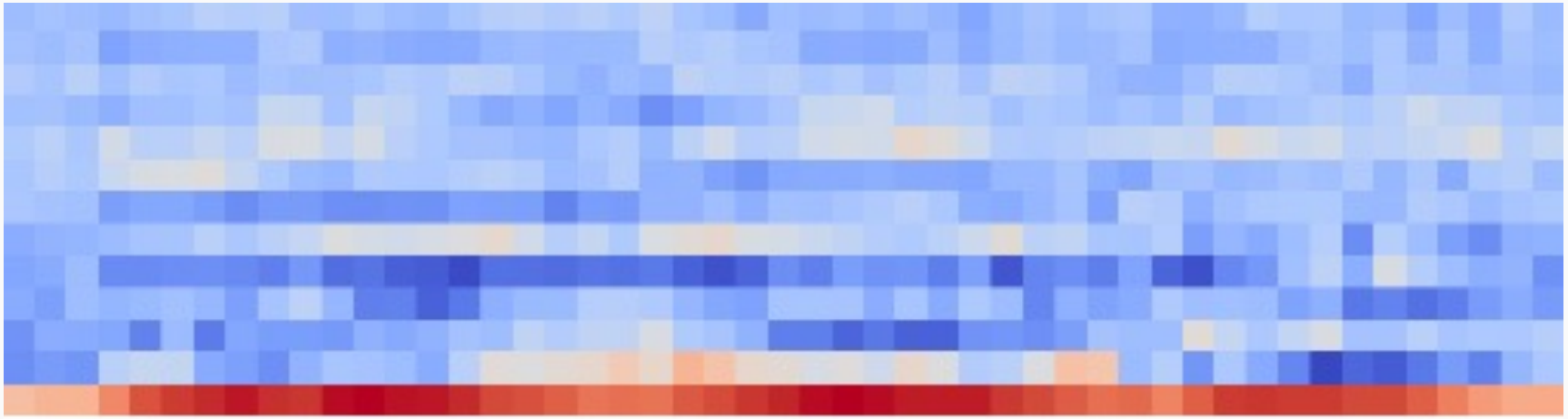
Mel Frequency Cepstral Coefficents (MFCCs) is a way of extracting features from an audio. The MFCC uses the MEL scale to divide the frequency band to sub-bands and then extracts the Cepstral Coefficents using Discrete Cosine Transform (DCT). MEL scale is based on the way humans distinguish between frequencies which makes it very convenient to process sounds.

Humans can hear roughly between 20Hz to 20KHz. The perception of sound is non-linear, and you can better distinguish between low frequency sounds than high frequency sounds e.g., humans can clearly hear the difference between 100Hz and 200Hz but not between 15kHz and 15.1kHz.

MFCC

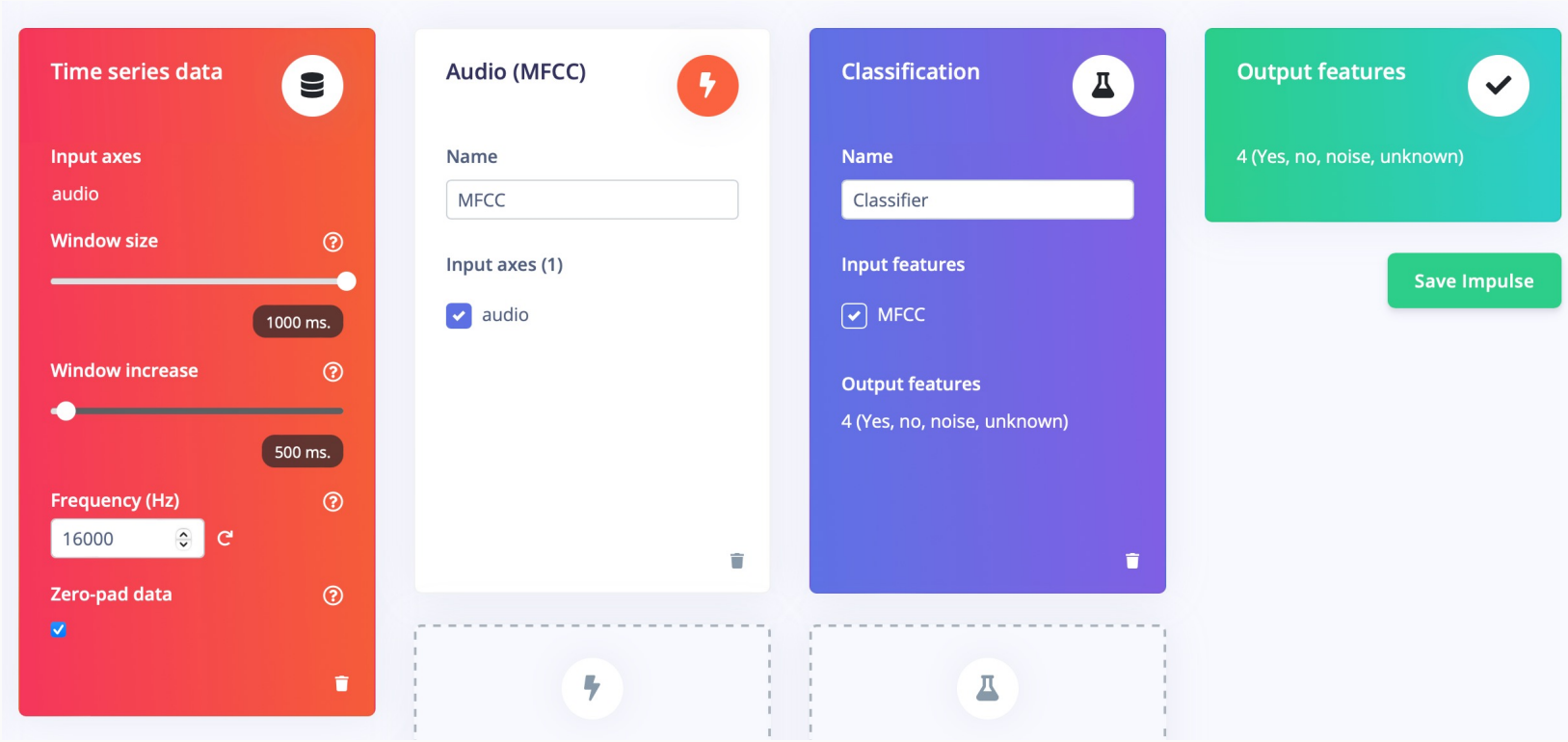
An **MFCC spectrogram** is a specially tuned spectrogram which highlights frequencies which are common in human speech.

In the spectrogram **the vertical axis represents the frequencies**, and the **horizontal axis represents time**. The patterns visible in a spectrogram contain information about what type of sound it represents. For example, the spectrogram in this image shows "Hello world":



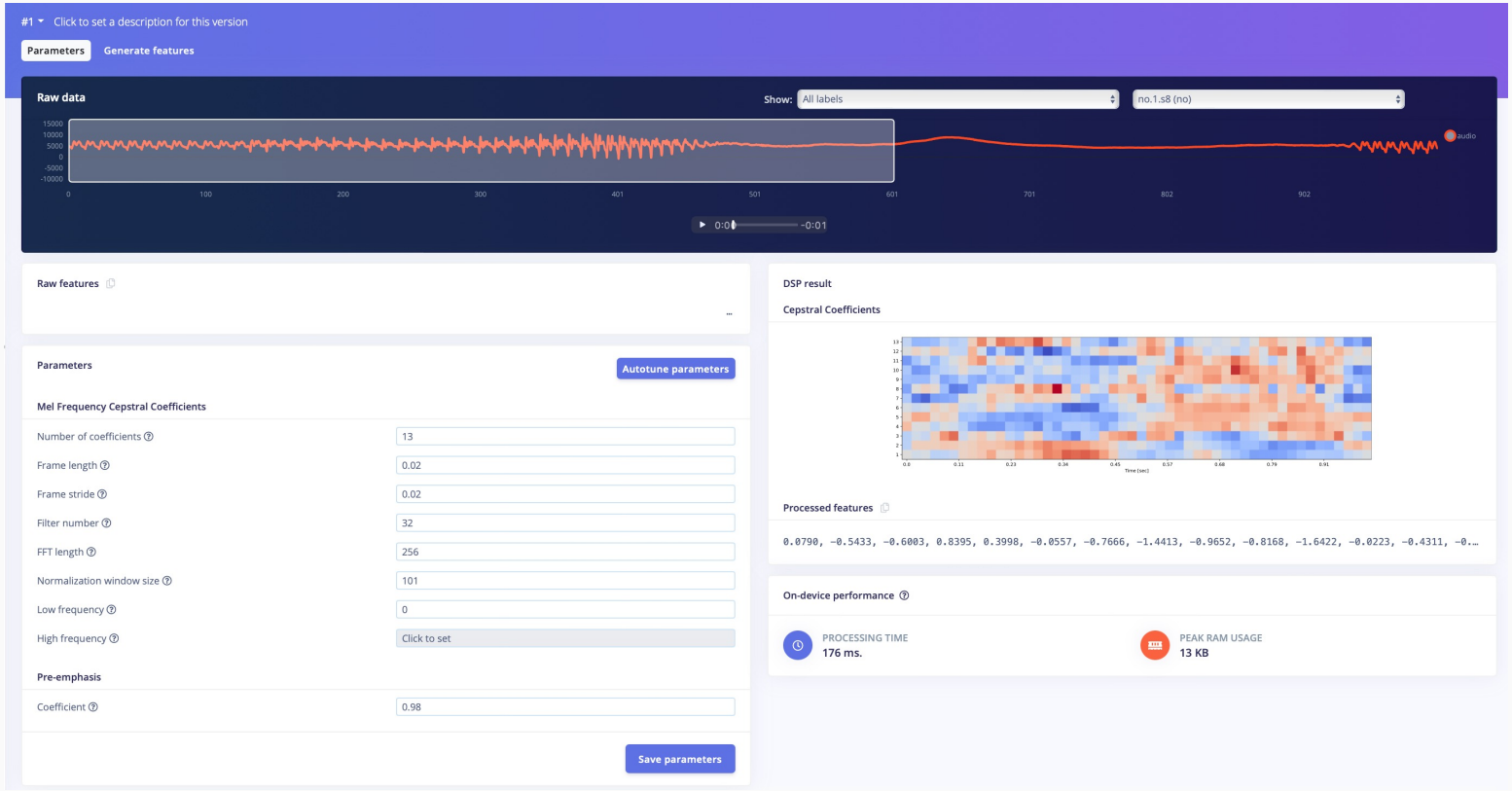
Lab

In the Add a learning block, select Classification. You can now save the Impulse.



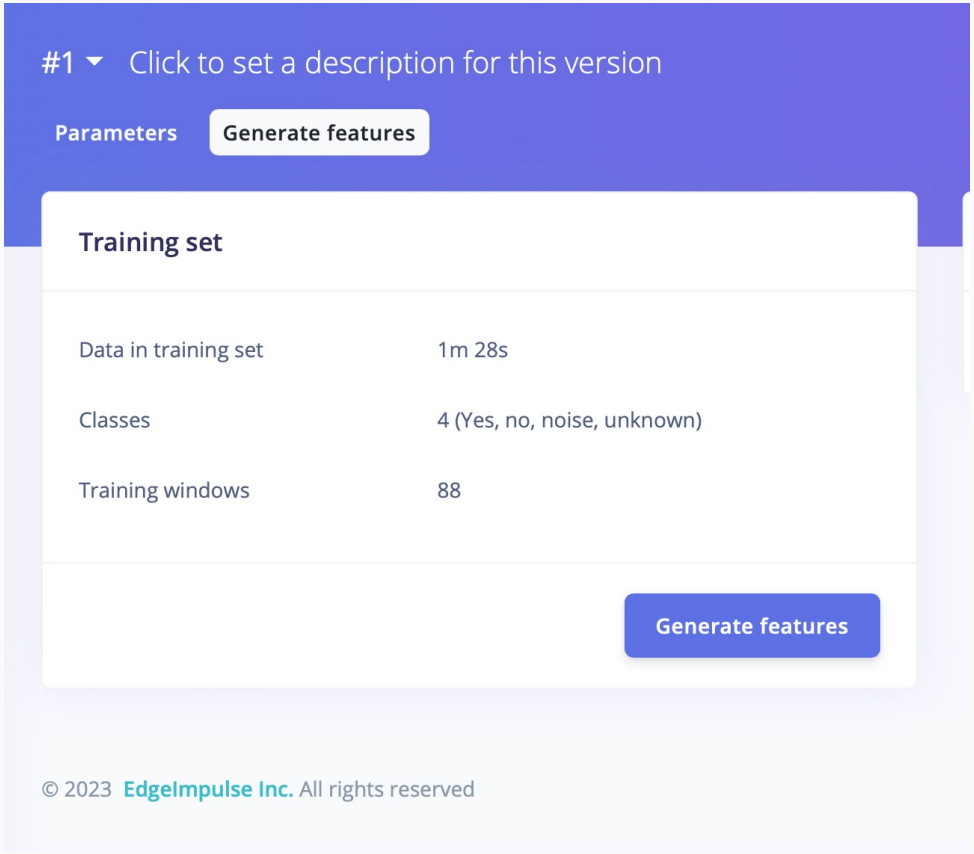
Lab

If you click on MFCC on the left side of the window, you should obtain the following information. Leave the default values and click on **Save Parameters.**



Lab

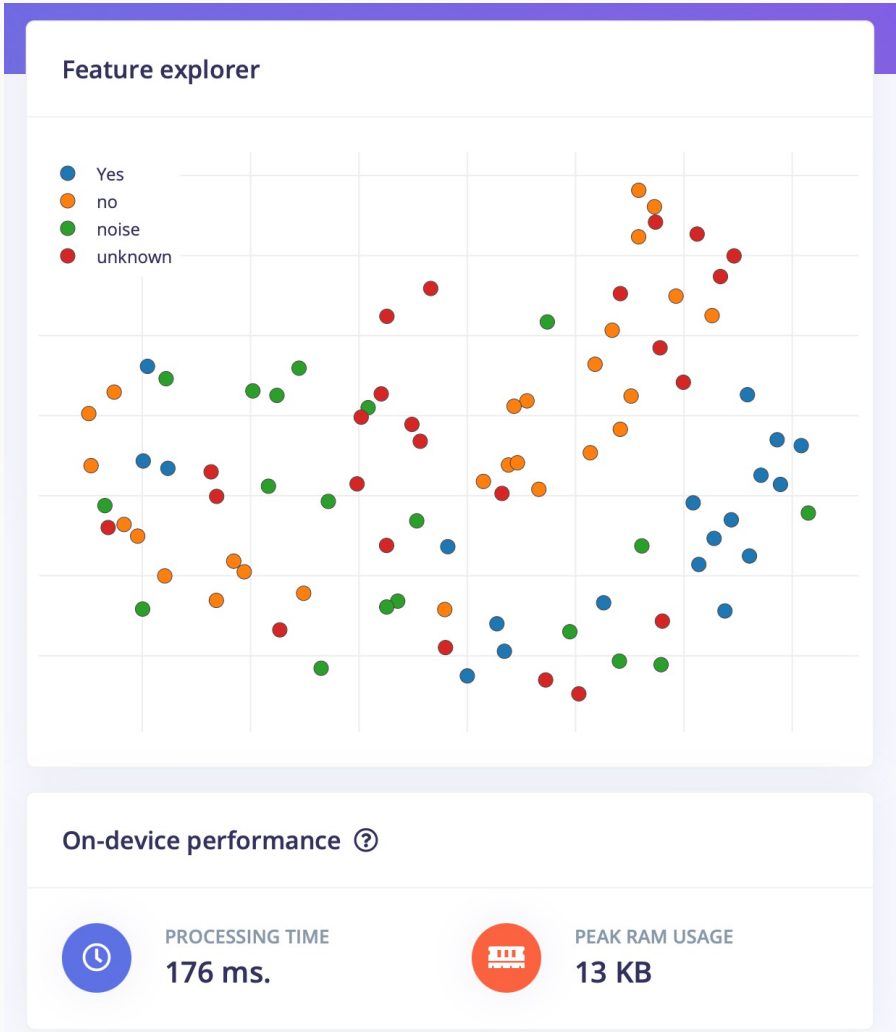
We will now generate the features. Click on Generate features.



Lab

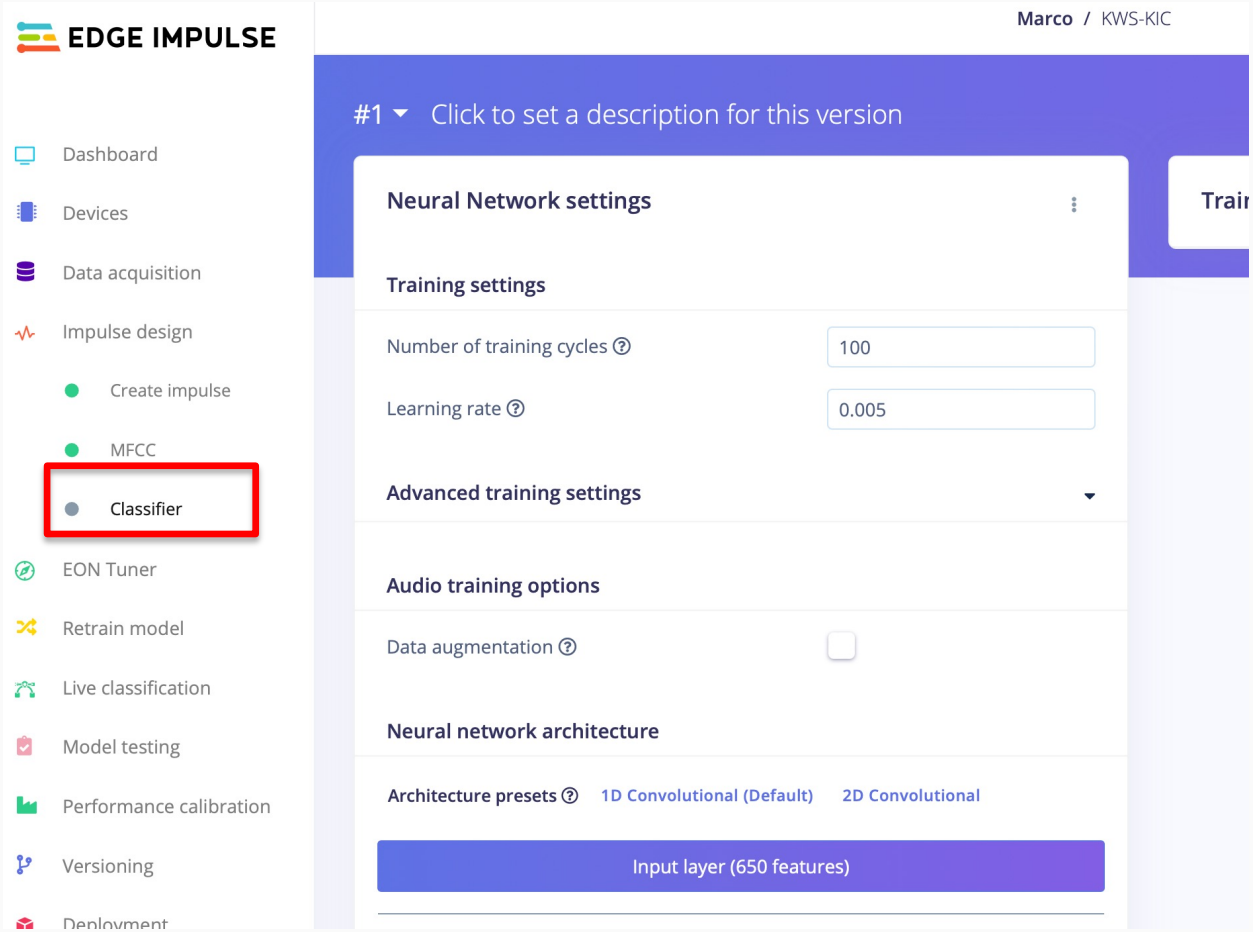
The Feature explorer should be organized in "clusters" of separated features. With our dataset the features are "mixed" and this is not a good sign.

The on-device performance part shows the processing time for inference and the peak ram usage. Both are low!



Lab

We are finally ready for Classification! Select Classifier on the left side of the window.



Neural Network

- It's time to start training a **neural network**. Neural networks are algorithms that can learn to recognize patterns that appear in their training data. The network that we're training here will take the MFCC as an input and try to map this to one of three classes—Yes, No, Noise or Unknown.
- A neural network is composed of layers of virtual "neurons. An input—in our case, an MFCC spectrogram—is fed into the first layer of neurons, which filters and transforms it based on each neuron's unique internal state. The first layer's output is then fed into the second layer, and so on, gradually transforming the original input into something radically different. In this case, the spectrogram input is transformed over four intermediate layers into just two numbers: the probability that the input represents your keyword, and the probability that the input represents 'noise' or 'unknown'.

Credit: https://docs.edgeimpulse.com/docs/tutorials/responding-to-your-voice

Neural Network

During training, the internal state of the neurons is gradually tweaked and refined so that the network transforms its input in just the right ways to produce the correct output. This is done by feeding in a sample of training data, checking how far the network's output is from the correct answer, and **adjusting the neurons' internal state to make it more likely that a correct answer is produced next time**. When done thousands of times, this results in a trained network.

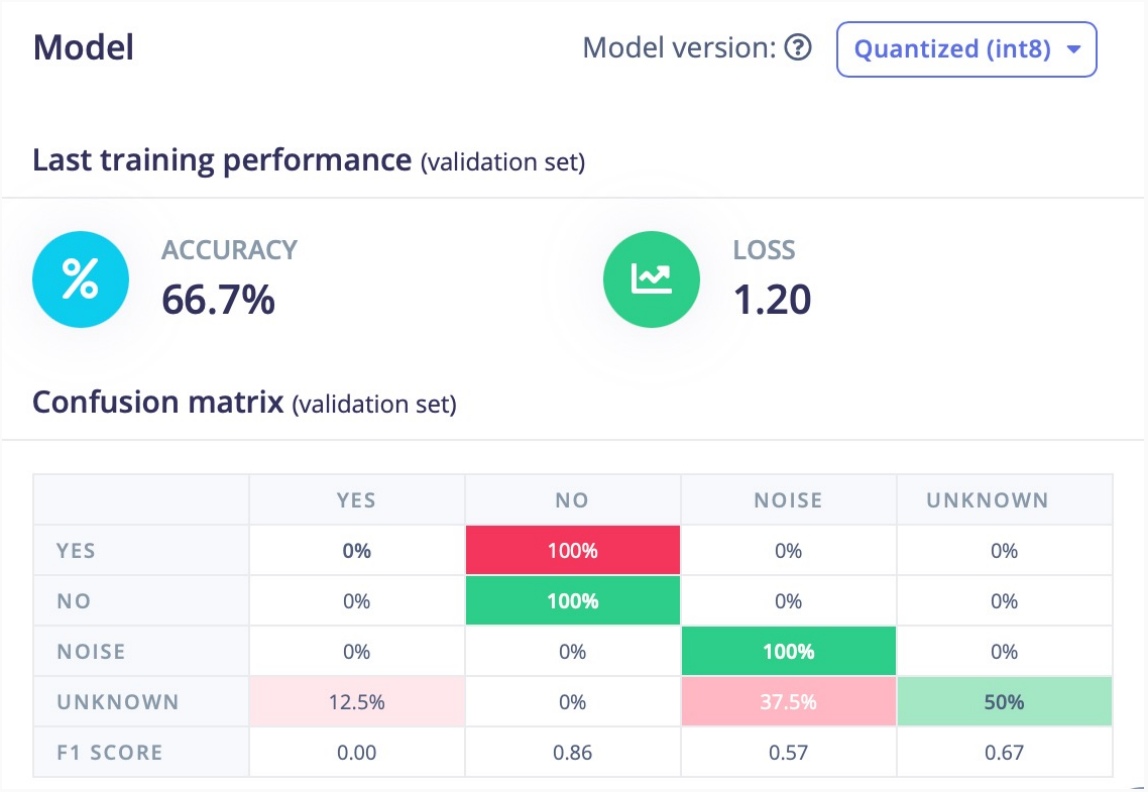
In our case, the basic architecture is defined with two blocks of Conv1D + MaxPooling (with 8 and 16 neurons, respectively) and a 0.25 Dropout. And on the last layer, after Flattening four neurons, one for each class.

As hyper-parameters, we will have a Learning Rate of 0.005 and a model that will be trained by 100 epochs.

Please select Data Augmentation.

Lab

The result is not great. Testing the model with the data put apart before training (Test Data), we got an accuracy of approximately 67%. → more data is needed!



Model Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 66.7%

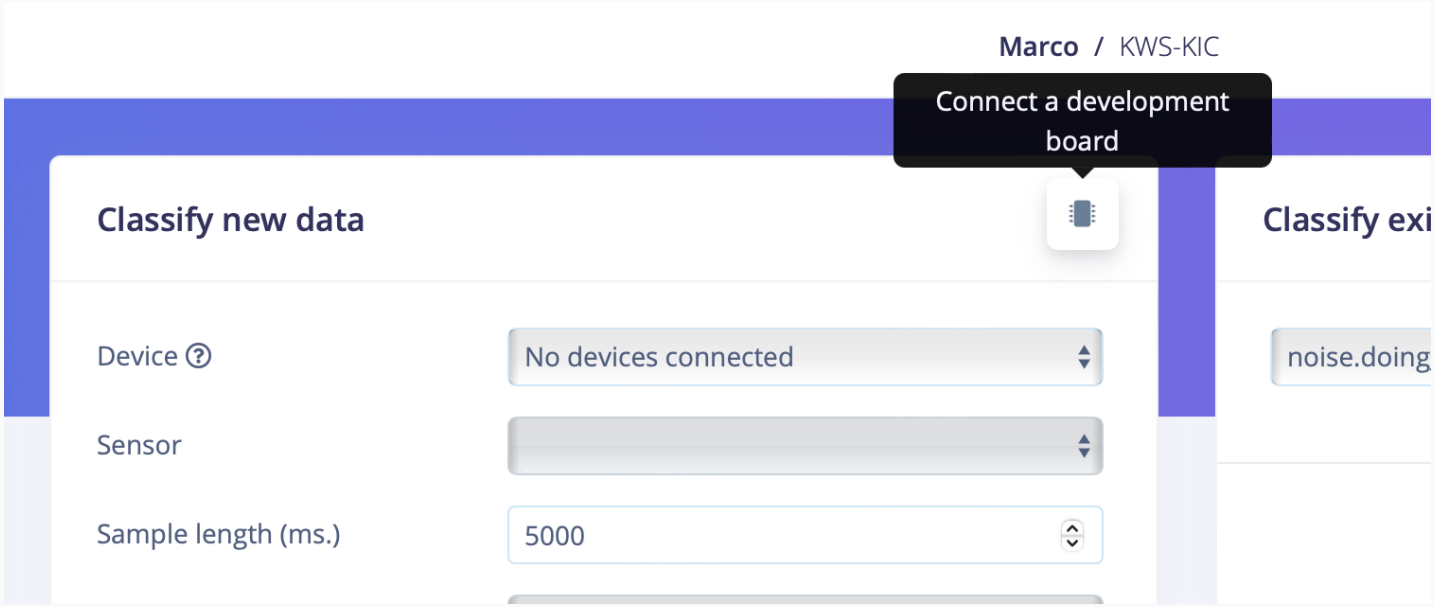
LOSS 1.20

Confusion matrix (validation set)

	YES	NO	NOISE	UNKNOWN
YES	0%	100%	0%	0%
NO	0%	100%	0%	0%
NOISE	0%	0%	100%	0%
UNKNOWN	12.5%	0%	37.5%	50%
F1 SCORE	0.00	0.86	0.57	0.67

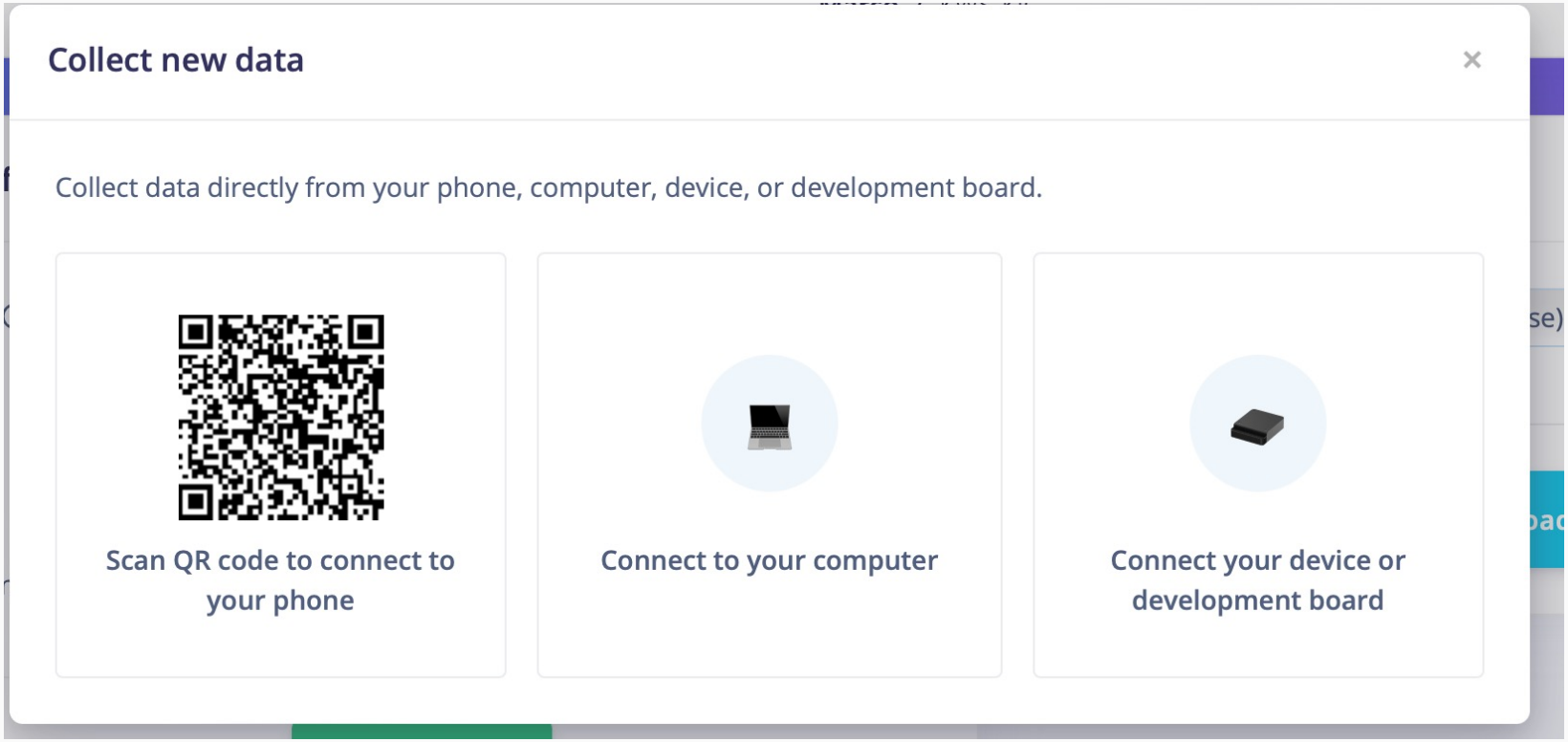
Lab

Before deployment of the model on the Nicla device, it is possible to perform Live Classification using a smartphone. Go to the **Live Classification** section and click on **Connect a Development board**:



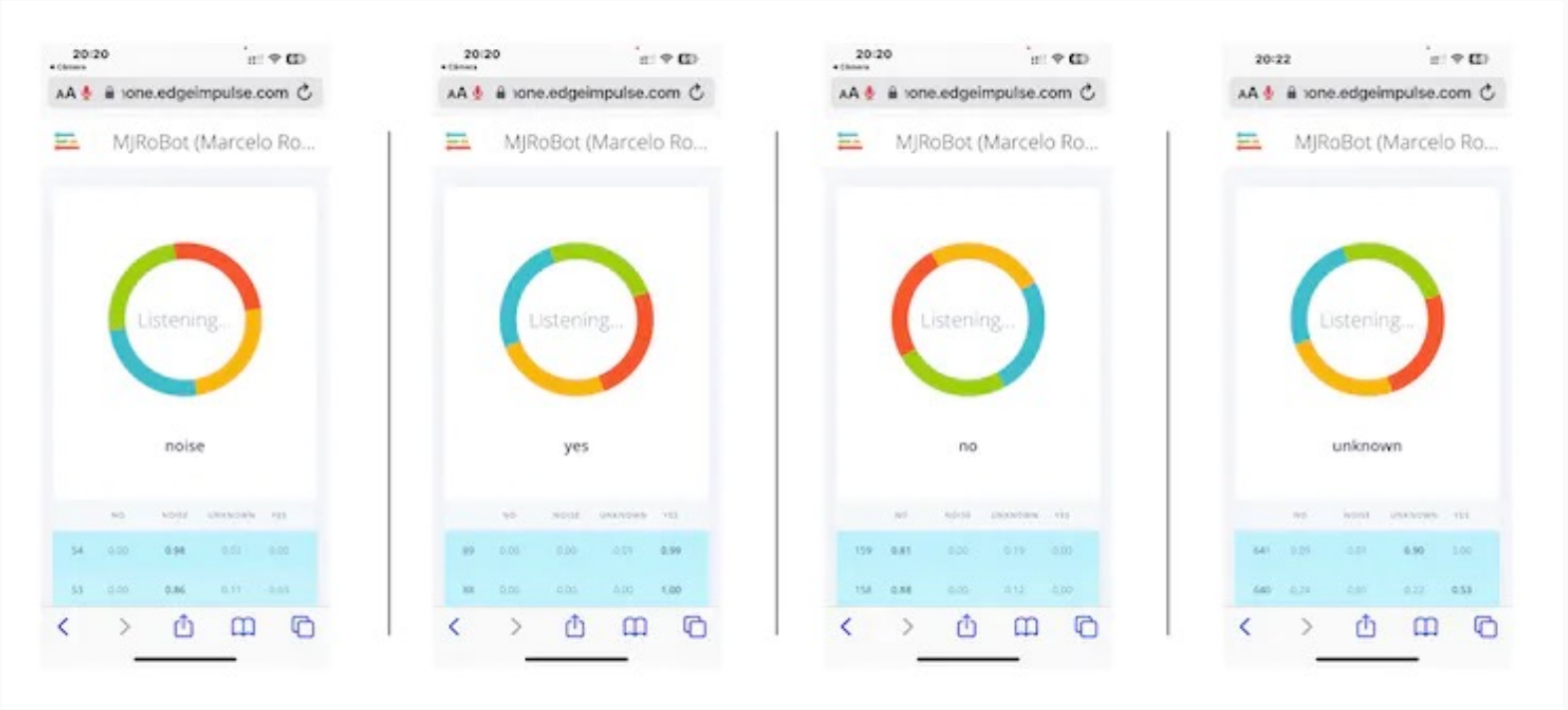
Lab

Point your phone to the QR code and select the link.



Lab

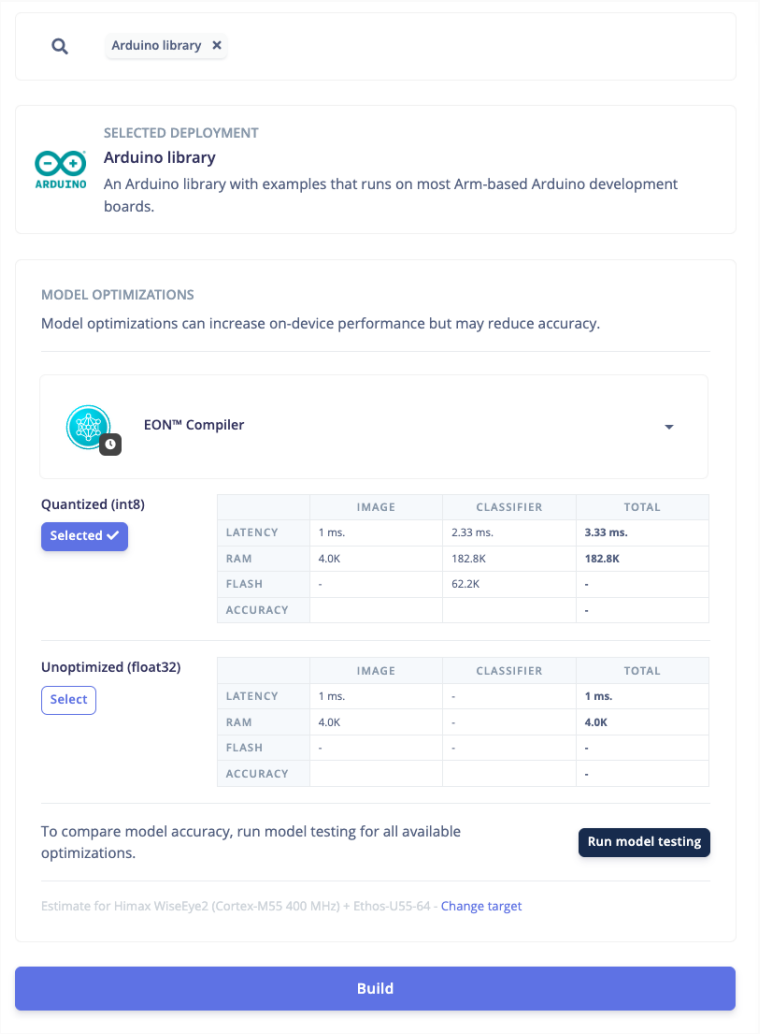
Your phone will be connected to Edge Impulse Studio. Select the option **Classification** on the app, and when it is running, start testing your keywords, confirming that the model is working with live and real data:



Lab

We now want to **deploy the model on the Nicla**. EdgeImpulse will package all the needed libraries, preprocessing functions, and trained models, downloading them to your computer.

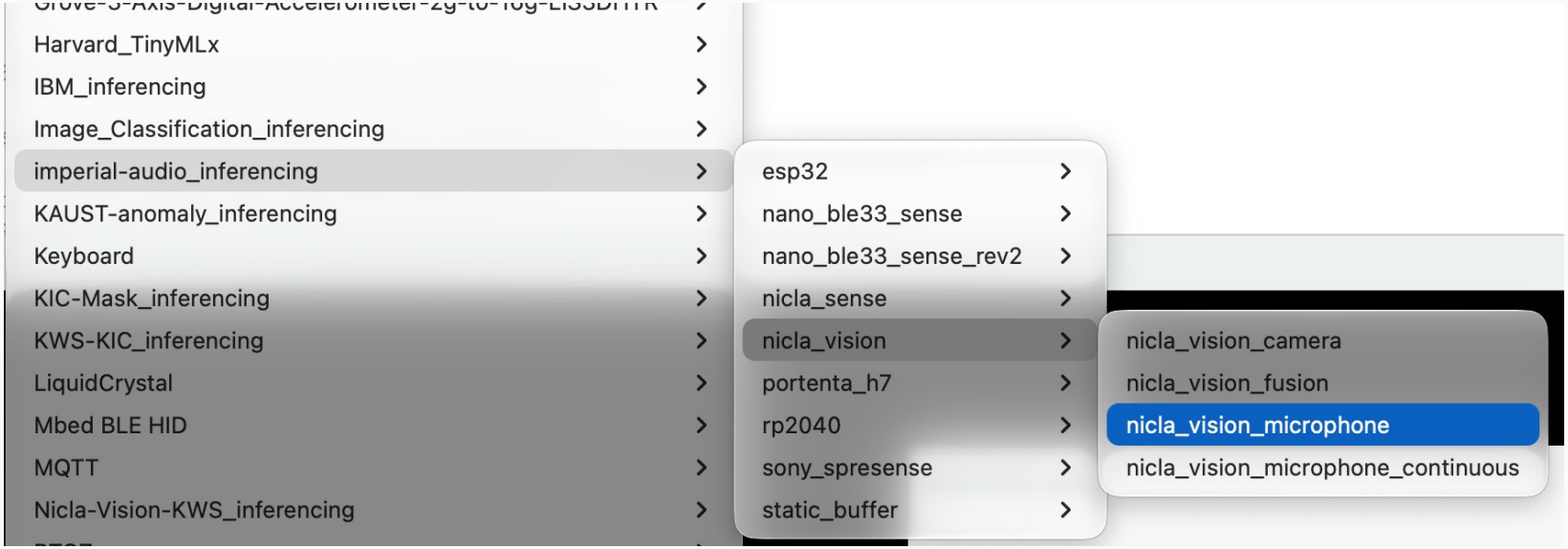
Select the Deployment tab in Edge Impulse, select the Arduino Library option, then choose Quantized (Int8) from the bottom menu and press Build.



Lab

Now it is time for a real test. We will make inferences fully disconnected from the Edge Impulse. Let's change one of the Nicla code examples created when you deploy the Arduino Library.

In your Arduino IDE, go to the File/Examples tab look for your project, and select nicla_vision_microphone:



Lab

Upload the code, Select Serial Monitor in the Arduino IDE and you should see:



Warm-up Task



Lab

Exercise:

Use an existing dataset (Yes/No or applause/no applause or …..) instead of gathering your data

Switch ON the LED when a specific word is detected

Primary Task



Lab

Exercise:

- Can you detect other sounds? Like the FAN on or off? How close should you be to the fan? Does it work when there is background noise?

Thanks!

TinyML whispers low,
Code dances on micro souls,
AI blooms in small.

mzennaro@ictp.it